

Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software

Juan José Medicis Zambrano

Proyecto de grado entregado para obtener el título de
Magister en Ingeniería de Software

Dirigida por
MsC Juan Camilo Parra Martinez

Pontificia Universidad Javeriana Cali
Facultad de Ingeniería y Ciencias
Maestría en Ingeniería de Software
Santiago de Cali
septiembre de 2026

Santiago de Cali, septiembre de 2026.

Señores
Pontificia Universidad Javeriana Cali.
Ph.D. Luisa Rincón
Directora Maestría en Ingeniería de Software.
Cali.

Cordial Saludo.

Por medio de la presente hago constar que en mi calidad de director de trabajo de grado he revisado el proyecto titulado "Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software" realizado por el estudiante de Magister en Ingeniería de Software Juan José Medicis Zambrano (cod: [CÓDIGO ESTUDIANTE]), el cual se encuentra terminado y considero que cumple con los requisitos para ser sustentado.
Atentamente,

MsC Juan Camilo Parra Martinez

Santiago de Cali, septiembre de 2026.

Señores
Pontificia Universidad Javeriana Cali
Ph.D. Luisa Rincón
Directora Maestría en Ingeniería de Software
Cali.

Cordial Saludo.

Me permito presentar a su consideración el proyecto de grado titulado "Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software" con el fin de cumplir con los requisitos exigidos por la Universidad y para que sea sometido a revisión del jurado y cumpla su aprobación, para conseguir posteriormente el título de Magister en Ingeniería de Software.

Atentamente,

Juan José Medicis Zambrano
Código: [CÓDIGO ESTUDIANTE]

Ficha Resumen
Trabajo de Grado Maestría en Ingeniería de Software

TÍTULO: Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software

1. Énfasis: Ingeniería de Software
2. Área de trabajo: Arquitectura de software, análisis estático de código y gobernanza técnica
3. Tipo de proyecto (Aplicado, Innovación, Investigación): Investigación aplicada
4. Estudiante: Juan José Medicis Zambrano
5. Correo electrónico: [COMPLETAR: correo institucional del estudiante]
6. Dirección y teléfono: [COMPLETAR: dirección y teléfono del estudiante]
7. Director: MsC Juan Camilo Parra Martinez
8. Vinculación del director: [COMPLETAR: Planta / Cátedra / Externo]
9. Correo electrónico del director: [COMPLETAR: correo institucional del director]
10. Co-Director (Si aplica): No aplica
11. Grupo o empresa que lo avala (Si aplica): No aplica
12. Otros grupos o empresas: No aplica
13. Palabras clave (al menos 5): deriva arquitectónica; análisis estático; grafos de dependencia; *Fitness Functions*; gobernanza arquitectónica; conformidad estructural
14. ODS que aplica al proyecto (Agenda 2030): ODS 9 (Industria, innovación e infraestructura)
15. Fecha de inicio: Mayo de 2026
16. Resumen: Este proyecto aborda la deriva arquitectónica, entendida como la divergencia progresiva entre la arquitectura prevista de un sistema de software y su implementación real, un fenómeno que incrementa el costo de mantenimiento y erosiona la calidad del producto. Su importancia radica en que las pequeñas y medianas empresas de software, que concentran buena parte del desarrollo en la industria, rara vez disponen de documentación arquitectónica actualizada ni de los recursos para adoptar herramientas de verificación sofisticadas, por lo que acumulan deuda técnica de manera silenciosa. Ante esta problemática, se propone diseñar,

construir y evaluar un mecanismo automatizado que detecta derivas arquitectónicas combinando el análisis estático de grafos de dependencia con la ejecución de *Fitness Functions*, definidas como verificaciones objetivas que cuantifican qué tan alineada está la implementación con las reglas estructurales declaradas. El objetivo general consiste en desarrollar y validar experimentalmente dicho mecanismo para clasificar el estado de conformidad estructural de un proyecto sin depender de documentación arquitectónica externa, mientras que los objetivos específicos abarcan la formulación de un catálogo de *Fitness Functions*, la implementación de un componente de análisis estático, la construcción de un motor de validación basado en una Tasa de Conformidad y la ejecución de un experimento controlado sobre un caso de estudio. La evaluación se realizó sobre cinco escenarios de un mismo sistema: una implementación de referencia y cuatro versiones alteradas, tres de ellas con una única deriva inducida para medir la sensibilidad de cada regla por separado. El mecanismo detectó la totalidad de las derivas y no produjo ningún hallazgo sobre la implementación de referencia, superando el criterio de éxito establecido. Los escenarios aislados revelaron, además, que la Tasa de Conformidad diluye las violaciones localizadas por ser una proporción no ponderada, lo que motivó incorporar la severidad del peor hallazgo como criterio complementario de clasificación. Entre las lecciones aprendidas destacan la viabilidad del enfoque con recursos limitados, la importancia de aislar las variables en el diseño experimental —fue esa decisión la que reveló la limitación de la métrica— y la necesidad de calibrar los umbrales de conformidad según el contexto de cada organización.

Resumen

La deriva arquitectónica es la divergencia progresiva entre la arquitectura prevista de un sistema y su implementación real, y constituye una de las principales fuentes de deuda técnica en las pequeñas y medianas empresas de software, que rara vez disponen de documentación arquitectónica actualizada o de recursos para adoptar plataformas de verificación sofisticadas. Este trabajo propone un mecanismo automatizado que detecta dicha deriva combinando el análisis estático de grafos de dependencia con la ejecución de *Fitness Functions*, entendidas como verificaciones objetivas que cuantifican la conformidad estructural de un proyecto. El mecanismo construye un grafo dirigido a partir del código compilado, evalúa tres dimensiones —integridad de capas, ausencia de ciclos y control de la concentración de dependencias salientes— y calcula una Tasa de Conformidad que clasifica el estado del sistema como aceptable, en observación o no conforme, acompañando cada hallazgo de la evidencia que lo origina. La propuesta se evaluó mediante un experimento controlado sobre cinco escenarios: una implementación de referencia y cuatro versiones alteradas, tres de ellas con una única deriva inducida para medir la sensibilidad de cada regla por separado. El mecanismo detectó la totalidad de las derivas sin producir hallazgos sobre la implementación de referencia. La evaluación reveló, además, que la Tasa de Conformidad, al ser una proporción no ponderada, diluye las violaciones aisladas y puede clasificar como aceptable un sistema con una desviación real; ese hallazgo motivó un refinamiento que incorpora la gravedad del peor hallazgo junto a la extensión que mide la métrica, con el cual los cinco escenarios pasaron a clasificarse conforme a lo esperado. Los resultados evidencian la viabilidad del enfoque para la gobernanza continua sin depender de documentación arquitectónica externa, y precisan los límites que toda métrica de conformidad agregada arrastra.

Palabras Clave: deriva arquitectónica, análisis estático, grafos de dependencia, *Fitness Functions*, gobernanza arquitectónica.

Abstract

Architectural drift is the progressive divergence between a system's intended architecture and its actual implementation, and it represents one of the main sources of technical debt in small and medium-sized software enterprises, which seldom keep architectural documentation up to date or can afford sophisticated verification platforms. This work proposes an automated mechanism that detects such drift by combining static dependency-graph analysis with the execution of *Fitness Functions*, understood as objective checks that quantify a project's structural conformance. The mechanism builds a directed graph from compiled code, evaluates three dimensions —layer integrity, absence of cycles, and control of outgoing dependency concentration— and computes a Conformance Rate that classifies the system's state as acceptable, under observation, or non-conformant, attaching to every finding the evidence that originates it. The proposal was evaluated through a controlled experiment over five scenarios: a reference implementation and four altered versions, three of which introduce a single drift each so that the sensitivity of every rule can be measured in isolation. The mechanism detected every induced drift without reporting any finding on the reference implementation. The evaluation further revealed that the Conformance Rate, being an unweighted proportion, dilutes isolated violations and may classify as acceptable a system that contains a genuine deviation; this finding motivated a refinement that combines the severity of the worst finding with the extent that the metric measures, under which all five scenarios were classified as expected. The results evidence the feasibility of the approach for continuous governance without relying on external architectural documentation, and delimit the constraints that any aggregate conformance metric carries.

Keywords: architectural drift, static analysis, dependency graphs, *Fitness Functions*, architectural governance.

Índice general

1. Introducción **1**
- 1.1. Antecedentes y contexto . . . 2
- 1.2. Definición del problema . . . 3
  - 1.2.1. Planteamiento del problema . . . 3
  - 1.2.2. Sistematización del problema . . . 3
- 1.3. Objetivos del proyecto . . . 4
  - 1.3.1. Objetivo general . . . 4
  - 1.3.2. Objetivos específicos . . . 4
- 1.4. Delimitaciones y alcances . . . 4
- 1.5. Justificación del trabajo de grado . . . 5
- 1.6. Metodología de la investigación . . . 5
- 1.7. Resultados obtenidos . . . 6
- 1.8. Estructura del documento . . . 7

2. Marco de referencia **9**
- 2.1. Marco Teórico . . . 9
  - 2.1.1. Arquitectura de software y atributos de calidad . . . 9
  - 2.1.2. Deriva y erosión arquitectónica . . . 9
  - 2.1.3. Deuda técnica arquitectónica y *smells* . . . 10
  - 2.1.4. *Fitness Functions* . . . 10
  - 2.1.5. Análisis estático y grafos de dependencia . . . 11
  - 2.1.6. Métricas de acoplamiento . . . 12
  - 2.1.7. Conformidad arquitectónica y gobernanza continua . . . 12
- 2.2. Estado del Arte . . . 13
- 2.3. Resumen del capítulo . . . 15

3. Desarrollo del Proyecto **17**
- 3.1. Descripción de la propuesta . . . 17
- 3.2. Requisitos . . . 18
- 3.3. Diseño de la propuesta . . . 19
- 3.4. Catálogo de *Fitness Functions* . . . 21
- 3.5. Operacionalización de la Tasa de Conformidad . . . 21
- 3.6. Decisiones de diseño . . . 23
- 3.7. Implementación de la propuesta . . . 24
  - 3.7.1. Exenciones declaradas . . . 25
- 3.8. Representación gráfica del grafo . . . 27
- 3.9. Interfaz gráfica . . . 28

3.10. Asistente para la definición del catálogo . . . 30
3.11. Verificación del prototipo . . . 32
3.12. Escenarios de evaluación . . . 33
3.13. Usos y escenarios de aplicación . . . 36
3.14. Trazabilidad entre requisitos e implementación . . . 38
3.15. Resumen del capítulo . . . 39

4. Evaluación **41**
4.1. Diseño de la evaluación . . . 41
4.1.1. Matriz de validación . . . 42
4.1.2. Criterios de éxito y procedimiento . . . 42
4.2. Resultados de la evaluación . . . 42
4.2.1. Contraste con la matriz de validación . . . 43
4.2.2. Sensibilidad por dimensión . . . 44
4.2.3. Trazabilidad del cálculo . . . 44
4.3. Un hallazgo sobre la métrica: la dilución de las violaciones localizadas . . . 45
4.4. Refinamiento del artefacto . . . 45
4.5. Contraste con los indicadores de la metodología . . . 46
4.6. Aplicación a un sistema externo . . . 47
4.7. Amenazas a la validez . . . 49
4.8. Resumen del capítulo . . . 52

5. Conclusiones **53**
5.1. Conclusiones . . . 53
5.2. Contribuciones del trabajo . . . 55
5.3. Trabajos futuros . . . 56
5.4. Lecciones aprendidas . . . 57

6. Riesgos éticos **59**
6.1. Riesgos éticos . . . 59
6.2. Principios orientadores . . . 59
6.3. Riesgos identificados y tratamiento aplicado . . . 59
6.4. Riesgos residuales . . . 61

Bibliografía **63**

A. Reproducción del experimento **67**
A.1. Entorno y dependencias . . . 67
A.2. Estructura del repositorio . . . 67
A.3. Pasos para reproducir el experimento . . . 68
A.4. Figuras de grafos y sistema externo . . . 68
A.5. Evidencia generada . . . 69

A.6. Determinismo de los resultados 69

Índice de figuras

3.1. Flujo del mecanismo de detección de deriva arquitectónica. El catálogo de reglas es un insumo externo: determina qué se considera admisible sin alterar el código del motor. 18

3.2. Módulos del prototipo y sentido de sus dependencias. Se omiten, por legibilidad, las aristas que saltan niveles —de la raíz a la interfaz y al servicio, del asistente al servicio, de la interfaz al reporte o hacia `grafo` y `config`—; ningún módulo depende de otro situado por encima de él, de modo que el grafo interno es acíclico. La interfaz, además, no accede a `grafo` ni a `config`: solo presenta lo que el servicio le entrega. . 20

3.3. Interfaz gráfica del prototipo analizando el escenario combinado (C4), descrito en la sección 3.12. La barra lateral guía los tres pasos —proyecto, catálogo y criterio de clasificación—; el panel principal muestra los indicadores, la conformidad por regla y el grafo, en el que se ha seleccionado el paquete `negocio`: lo que no le concierne se atenúa y la ficha de la derecha detalla su capa, sus acoplamientos, sus vecinos y los hallazgos en que interviene. 29

3.4. Asistente de catálogo sobre el escenario C4. A la izquierda, la configuración del modelo y lo que se envía; en el centro, la conversación, con la deriva probable y las preguntas para la persona; a la derecha, el catálogo propuesto, editable y validado, y su previsualización sobre el código, que reproduce el diagnóstico de la tabla 4.2. La respuesta del modelo se simuló para la captura. 31

3.5. Grafo de dependencias de la implementación de referencia (C0), dibujado por el propio mecanismo: 4 paquetes y 5 dependencias, sin violaciones. Cada banda corresponde a una capa declarada; el dominio compartido, que no pertenece a ninguna, ocupa la columna lateral. 34

3.6. Grafo del escenario C2, dibujado por el propio mecanismo. El ciclo queda contenido en la capa de negocio: ambos paquetes comparten fila y su dependencia mutua se traza como dos arcos, uno por encima y otro por debajo. Ninguna arista remonta la estratificación. 35

3.7. Grafo del escenario combinado (C4), dibujado por el propio mecanismo: 5 paquetes y 11 dependencias. Las dos dependencias que la persistencia dirige hacia capas superiores violan la estratificación y rodean los nodos por fuera; el ciclo que cierran se marca en los tres paquetes de capa, y el paquete que concentra dependencias salientes aparece en ámbar en la columna lateral. 36

4.1. Grafo de dependencias de *ArchUnit* 1.3.0 con las exenciones declaradas, dibujado por el propio mecanismo a partir del archivo `.jar`. Las bandas siguen la estratificación hipotética `library` → `lang` → `core` → `base`; el núcleo de anotaciones exento ocupa la columna lateral y sus dependencias se dibujan atenuadas. 50

Índice de tablas

2.1. Comparación de los trabajos relacionados con la propuesta. Leyenda: ✓ lo cumple, ∼ lo cumple parcialmente, × no lo cumple. 14

3.1. Catálogo de *Fitness Functions* implementadas. 22
3.2. Trazabilidad entre requisitos, implementación y evidencia de cumplimiento. 39

4.1. Matriz de validación: comportamiento esperado de cada *Fitness Function* por escenario. 42
4.2. Resultados agregados del mecanismo sobre los cinco escenarios. 43
4.3. Contraste entre el comportamiento esperado y el observado. Cada celda indica el número de hallazgos reportados por la regla. 43
4.4. Hallazgos detectados en el escenario combinado (C4). 43
4.5. Conformidad local por *Fitness Function* en cada escenario (verificaciones superadas sobre verificaciones realizadas). 44
4.6. Clasificación de los cinco escenarios bajo ambos criterios. La columna «esperado» recoge el estado que el escenario merece según las derivas efectivamente inducidas. . 46
4.7. Contraste entre los indicadores metodológicos y los resultados observados. 47
4.8. Diagnóstico de *ArchUnit* 1.3.0 antes y después de declarar las exenciones. 48

5.1. Cumplimiento de los criterios de éxito comprometidos en el anteproyecto. 54

6.1. Riesgos éticos, tratamiento aplicado y estado al cierre del proyecto. 61

Capítulo 1

Introducción

La arquitectura de un sistema de software no es una propiedad estática: se moldea, se negocia y, con frecuencia, se deteriora a medida que el código evoluciona. Cuando las decisiones de implementación se apartan de la estructura prevista sin que nadie lo registre, aparece la deriva arquitectónica, un distanciamiento paulatino entre la arquitectura deseada y la que efectivamente quedó plasmada en el código fuente (Martín et al., 2024). Se trata de un fenómeno silencioso, acumulativo y costoso, que rara vez se manifiesta de golpe; por el contrario, avanza mediante pequeñas concesiones —una dependencia que cruza una capa, un atajo entre módulos, una clase que concentra demasiadas responsabilidades— que, tomadas de forma aislada, parecen inofensivas.

Las consecuencias de ese deterioro han sido ampliamente documentadas. La erosión arquitectónica incrementa de forma sostenida el esfuerzo de mantenimiento, encarece la incorporación de nuevas funcionalidades y reduce la capacidad del equipo para predecir el impacto de sus cambios (Pigazzini, 2021). Estudios empíricos recientes confirman que ciertos *smells* arquitectónicos correlacionan de manera negativa con la mantenibilidad del software (Jolak et al., 2025), y que los propios desarrolladores perciben la deriva como una fuente persistente de fricción en su trabajo cotidiano (Anthony et al., 2024). El problema no es menor: se estima que una proporción considerable del presupuesto destinado a tecnologías de la información se consume en actividades de mantenimiento, y que la erosión arquitectónica puede elevar el costo de evolución de un sistema en porcentajes de dos dígitos (Pigazzini, 2021).

El escenario se agudiza en las pequeñas y medianas empresas (PyMES) de software. A diferencia de las grandes organizaciones, estas compañías suelen operar con equipos reducidos, ciclos de entrega ajustados y una documentación arquitectónica escasa, desactualizada o, en muchos casos, inexistente (Hamed et al., 2023). Carecen, además, de los recursos humanos y económicos para desplegar las plataformas de verificación sofisticadas que sí están al alcance de corporaciones con mayor madurez. El resultado es una paradoja preocupante: precisamente donde la gobernanza técnica sería más rentable, es donde menos se practica. Frente a este panorama, resulta pertinente preguntarse si es posible trasladar los beneficios de la verificación arquitectónica a un entorno de recursos limitados, mediante un mecanismo ligero, automatizado y que no dependa de documentación externa.

Este trabajo propone responder esa inquietud a través de un mecanismo que combina el análisis estático de grafos de dependencia con la ejecución de *Fitness Functions*. Estas últimas, popularizadas en la práctica de la ingeniería de software, consisten en verificaciones objetivas y automatizables que expresan, en forma de criterios cuantificables, las propiedades que la arquitectura debe preservar (Paul and Wang, 2019; Continuous Architecture, 2026). En lugar de confiar en un documento de arquitectura que probablemente esté desactualizado, el mecanismo infiere la estructura real del sis-

tema a partir de su propio código, la contrasta contra un conjunto acotado de reglas estructurales y emite un diagnóstico sobre el estado de conformidad del proyecto. De esta forma, la gobernanza arquitectónica deja de ser un ejercicio documental y se convierte en una práctica continua, verificable y de bajo costo.

1.1. Antecedentes y contexto

La preocupación por mantener la coherencia entre el diseño previsto y el código construido no es reciente. Desde que la ingeniería de software reconoció a la arquitectura como una disciplina con identidad propia, a comienzos de la década de 1990, quedó establecido que la estructura de un sistema condiciona de manera decisiva sus atributos de calidad: rendimiento, mantenibilidad, seguridad y capacidad de evolución (Perry and Wolf, 1992; Bass et al., 2012). La arquitectura dejó de entenderse como un dibujo accesorio y pasó a concebirse como el andamiaje sobre el que se sostienen todas las decisiones posteriores; de ahí que su deterioro no sea un asunto cosmético, sino una amenaza concreta contra la viabilidad del producto.

Paralelamente, la evidencia empírica consolidó una constatación incómoda: los sistemas reales cambian, y cambian mucho. Las leyes de evolución del software enunciadas por Lehman (1980) anticiparon que todo programa que resulte útil a su comunidad se verá sometido a una presión continua de modificación, y que, a medida que crece, su estructura tiende a degradarse a menos que se invierta un esfuerzo explícito en preservarla. La deriva arquitectónica es, en buena medida, la manifestación contemporánea de esa ley: el costo de no intervenir se paga, tarde o temprano, en forma de complejidad estructural y deuda acumulada.

El concepto de deuda técnica, acuñado por Cunningham (1992) para describir las consecuencias de aplazar decisiones de diseño, proporcionó el vocabulario con el que la industria pudo nombrar el fenómeno. Desde entonces, la gestión de esa deuda se convirtió en un campo de estudio activo, y su variante estructural —la deuda técnica arquitectónica— en uno de sus frentes más difíciles de atender, precisamente porque sus síntomas no se manifiestan en una línea de código aislada, sino en la relación entre componentes que, por separado, parecen saludables.

En el ámbito de las pequeñas y medianas empresas, esta problemática se cruza con otra restricción. La literatura sobre transformación digital de PyMES coincide en señalar que estas organizaciones necesitan soluciones de bajo costo, de adopción incremental y que no exijan especialistas dedicados (Patrício and Varela, 2025; Jamil, 2026). Los marcos de gobernanza pensados para corporaciones resultan, por su peso y su demanda de roles, poco realistas para equipos pequeños. Esta tensión entre la necesidad de control estructural y la escasez de recursos constituye el terreno fértil sobre el que se levanta la presente propuesta.

1.2. Definición del problema

1.2.1. Planteamiento del problema

El punto de partida es una situación concreta y, a la vez, extendida: los sistemas de software de las PyMES crecen sin que exista un mecanismo que vigile, de manera continua, la coherencia entre lo que se pretendía construir y lo que realmente se construyó. Los síntomas son reconocibles para cualquier profesional del área: dependencias que cruzan fronteras que no debían cruzarse, módulos que se acoplan de forma imprevista, componentes que se vuelven imprescindibles para todo el sistema y una dificultad creciente para estimar cuánto costará el siguiente cambio. Las causas son múltiples, pero se concentran en la ausencia de documentación arquitectónica viva, la presión por entregar con rapidez y la falta de herramientas de verificación asequibles para equipos pequeños (Hamed et al., 2023; Klein et al., 2022).

Si la situación se mantiene, los efectos negativos se profundizan. La deuda técnica se acumula en forma de *smells* arquitectónicos que, según la evidencia disponible, terminan comprometiendo la mantenibilidad y la longevidad de los productos (Sas and Avgeriou, 2022; Jolak et al., 2025). Cada desviación no detectada encarece las siguientes: el equipo invierte tiempo en comprender un código cuya estructura real ya no se corresponde con ninguna representación mental compartida. La organización, en consecuencia, ve erosionada su capacidad de respuesta ante el mercado, justo cuando más la necesita.

1.2.2. Sistematización del problema

A partir de lo anterior, la investigación se orienta mediante una pregunta central, formulada de manera abierta para que actúe como guía y no como una búsqueda de una única causa:

¿Es posible diseñar y evaluar un mecanismo automatizado, basado en análisis estático, grafos de dependencia y Fitness Functions, que permita detectar tempranamente derivas arquitectónicas y clasificar el estado de conformidad estructural de un sistema de software para PyMES sin depender de documentación arquitectónica externa?

De esta pregunta se desprenden interrogantes subsidiarios que delimitan las etapas del trabajo: qué propiedades estructurales deben vigilarse para que el diagnóstico sea útil y acotado; de qué manera puede obtenerse, a partir del código fuente, un grafo de dependencias lo bastante fiel para soportar la evaluación; cómo debe traducirse el resultado de las verificaciones en una clasificación del estado arquitectónico que sea comprensible para un equipo de desarrollo; y, finalmente, cómo evidenciar, mediante un experimento controlado, que el mecanismo efectivamente detecta las derivas que se le inducen.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Desarrollar y evaluar experimentalmente un mecanismo automatizado para la detección de derivas arquitectónicas, basado en grafos de dependencia y en la ejecución de *Fitness Functions*, que permita clasificar el estado de conformidad estructural de proyectos de software de PyMES sin depender de documentación arquitectónica externa.

1.3.2. Objetivos específicos

- **OBJ1.** Formular un catálogo acotado de *Fitness Functions* que traduzcan, en criterios verificables, las propiedades estructurales esenciales que deben preservarse en un sistema de software (integridad de capas, ausencia de ciclos y control de la concentración de dependencias salientes).
- **OBJ2.** Implementar un componente de análisis estático capaz de construir, a partir del código fuente, un grafo de dependencias entre paquetes que sirva de insumo común para todas las verificaciones.
- **OBJ3.** Construir un motor de validación que ejecute las *Fitness Functions* sobre el grafo, agregue sus resultados en una Tasa de Conformidad (CR) y clasifique el estado arquitectónico del sistema en niveles interpretables.
- **OBJ4.** Ejecutar un experimento controlado sobre un caso de estudio para comprobar que el mecanismo detecta las derivas inducidas y clasifica correctamente tanto una implementación de referencia como una versión alterada.

1.4. Delimitaciones y alcances

El trabajo se circunscribió a un conjunto de fronteras definidas de antemano, con el propósito de hacer viable la propuesta en el tiempo y con los recursos disponibles. En primer lugar, el mecanismo se orientó exclusivamente al ecosistema Java, aprovechando la madurez de las herramientas de análisis estático disponibles para esa plataforma; no se abordaron otros lenguajes ni entornos de ejecución. En segundo lugar, la detección se limitó a tres dimensiones estructurales —integridad de capas, dependencias cíclicas y concentración de dependencias salientes—, dejando por fuera otras manifestaciones de la deriva, como la erosión a nivel de servicios distribuidos o las desviaciones en los patrones de diseño. En tercer lugar, el grafo de dependencias se construyó a nivel de paquetes: las violaciones se clasifican con esa granularidad, si bien la evidencia que las acompaña desciende hasta la referencia concreta entre clases que las origina. Por último, la validación se realizó sobre escenarios sintéticos controlados, sin desplegar el mecanismo en un entorno productivo real; la generalización a organizaciones con arquitecturas heterogéneas quedó, por tanto, planteada como trabajo futuro y no como parte del alcance cumplido.

1.5. Justificación del trabajo de grado

Resolver el problema planteado aporta valor en varios frentes que se refuerzan entre sí. Desde la perspectiva económica, un mecanismo que detecte la deriva de forma temprana permite contener el crecimiento del costo de mantenimiento, que hoy consume una fracción desproporcionada del presupuesto tecnológico de las empresas y que la erosión arquitectónica tiende a agravar (Pigazzini, 2021). Anticipar la detección equivale a corregir las desviaciones cuando todavía son baratas de revertir, en lugar de pagar su factura cuando ya comprometen la viabilidad del producto.

Desde el punto de vista técnico, la propuesta traslada a las PyMES una capacidad que hasta ahora se concentraba en organizaciones de mayor tamaño: la verificación continua de la conformidad arquitectónica. La evidencia disponible subraya la importancia de esta práctica para sostener la calidad de los sistemas a lo largo del tiempo (Klein et al., 2022), y herramientas como *ArchUnit* ya demostraron que es posible expresar reglas arquitectónicas como pruebas ejecutables en Java (TNG Technology Consulting, 2026). Este trabajo aprovecha esa base conceptual, pero la orienta hacia un diagnóstico agregado y clasificatorio, más sencillo de interpretar para un equipo sin especialistas dedicados a la arquitectura.

En el plano social y organizacional, la solución empodera a los equipos de desarrollo, a los arquitectos y a los líderes técnicos de las PyMES, quienes pasan de depender de la intuición o de documentos obsoletos a disponer de una señal objetiva y reproducible sobre el estado estructural de sus proyectos. La propuesta se alinea, además, con las iniciativas de gobernanza técnica liviana que vienen cobrando fuerza en el ecosistema de las pequeñas organizaciones, en las que los marcos de control deben ser proporcionales a los recursos disponibles (Modawski and Wolniak, 2025).

1.6. Metodología de la investigación

La investigación se desarrolló bajo el enfoque de *Design Science Research* (DSR), una estrategia especialmente adecuada cuando el propósito no es solo comprender un fenómeno, sino también diseñar y validar un artefacto que lo aborde (Hevner et al., 2004). El proceso siguió las seis actividades que estructuran la metodología (Peffers et al., 2007): identificación del problema, definición de los objetivos de la solución, diseño y desarrollo del artefacto, demostración sobre un caso, evaluación frente a criterios explícitos y comunicación de los resultados. El ciclo no fue lineal: la evaluación reveló una limitación de la métrica de conformidad que obligó a refinar el criterio de clasificación y a reejecutar el experimento, de modo que el artefacto que se presenta es el producto de una segunda iteración y no de un único paso de construcción.

Metodológicamente, el trabajo articuló procedimientos de distinta naturaleza. La construcción del grafo de dependencias se apoyó en técnicas de análisis estático, que extraen la estructura del sistema sin necesidad de ejecutarlo. La evaluación de las propiedades estructurales se formalizó mediante un catálogo de *Fitness Functions*, cada una de las cuales define sus propios criterios de verificación e incumplimiento. Finalmente, la agregación de resultados se resolvió con una métrica de conformidad acompañada de umbrales de clasificación, lo que permitió traducir los hallazgos individuales en un veredicto global sobre el estado del sistema. La validación se realizó mediante

un experimento controlado que contrastó una implementación de referencia contra cuatro versiones deliberadamente alteradas, tres de ellas con una única deriva inducida para poder medir la sensibilidad de cada regla por separado, siguiendo las convenciones de diseño y reporte habituales en la investigación empírica en ingeniería de software (Wohlin et al., 2012; Runeson and Höst, 2009).

Conviene explicitar la correspondencia entre los objetivos específicos y las fases del ciclo DSR, pues esa articulación otorga trazabilidad al trabajo. OBJ1 —la formulación del catálogo de *Fitness Functions*— corresponde a la fase de diseño del artefacto, en la que se tradujeron las propiedades estructurales deseadas en criterios verificables. OBJ2 —la construcción del grafo de dependencias— y OBJ3 —la construcción del motor de validación— materializan la fase de desarrollo, en la que el diseño se concretó como un prototipo ejecutable. OBJ4 —el experimento controlado— constituye la fase de evaluación, en la que el artefacto se sometió a una prueba de sensibilidad diseñada para evidenciar, de manera objetiva, si era capaz de detectar las derivas inducidas. Los resultados de esa evaluación retroalimentaron el artefacto y obligaron a refinar su criterio de clasificación, cerrando así el ciclo iterativo que caracteriza a la investigación en diseño. Esta estructura metodológica no solo ordenó el trabajo, sino que permitió distinguir con nitidez, en cada etapa, aquello que se había logrado de aquello que quedaba pendiente de verificación.

1.7. Resultados obtenidos

El mecanismo desarrollado logró construir, a partir del código compilado, un grafo de dependencias entre paquetes y ejecutar sobre él las tres *Fitness Functions* previstas. Sobre la implementación de referencia obtuvo una Tasa de Conformidad del cien por ciento, sin un solo hallazgo, y la clasificó como *aceptable*. Sobre las cuatro versiones alteradas detectó la totalidad de las seis derivas inducidas —rupturas de la integridad de capas, dependencias cíclicas y concentración excesiva de dependencias salientes—, superando el criterio de éxito del ochenta por ciento fijado en el anteproyecto. En el escenario que combina las tres derivas, la conformidad descendió al cincuenta y siete por ciento y el sistema fue clasificado como *no conforme*.

La evaluación aportó, además, un hallazgo que no se había anticipado en su magnitud. Los escenarios construidos con una única deriva revelaron que la Tasa de Conformidad, al ser una proporción no ponderada, diluye las violaciones aisladas: en dos de esos escenarios el mecanismo detectó correctamente la desviación pero la métrica se mantuvo por encima del umbral de aceptación, de modo que el sistema quedaba clasificado como aceptable pese a contener una violación real. Ese resultado motivó un refinamiento del criterio de clasificación —que incorpora la gravedad del peor hallazgo junto a la extensión que mide la métrica, sin alterar su cálculo— con el cual los cinco escenarios pasaron a clasificarse conforme a lo esperado.

Al margen del experimento controlado, el mecanismo se aplicó sobre una biblioteca de terceros de casi mil clases para observar su comportamiento ante código que nadie preparó para él. El análisis completo tardó poco más de un segundo y la regla de integridad de capas no produjo un solo falso positivo. Reveló, en cambio, una segunda limitación: sobre código real, una regla puramente topológica confunde un componente deteriorado con uno que concentra dependencias por el papel que desempeña, hasta el punto de que dos tercios de sus hallazgos resultaron ser ruido.

La respuesta fue incorporar exenciones declaradas y registradas en el reporte, que eliminan ese ruido sin silenciar los hallazgos genuinos. Los capítulos de desarrollo y evaluación detallan los resultados y el razonamiento que condujo a ambos ajustes.

1.8. Estructura del documento

El presente documento se organiza en seis capítulos que reflejan la secuencia lógica de la investigación. El capítulo siguiente, *Marco de referencia*, expone las bases teóricas sobre deriva arquitectónica, *Fitness Functions*, análisis estático, grafos de dependencia y métricas de acoplamiento, junto con el estado del arte de los trabajos relacionados. El capítulo *Desarrollo del proyecto* describe el diseño y la implementación del mecanismo, incluidos el catálogo de reglas, la operacionalización de la métrica de conformidad y los escenarios de evaluación construidos. El capítulo *Evaluación* presenta el diseño experimental, la matriz de validación, los resultados obtenidos y el refinamiento que estos motivaron, junto con las amenazas a la validez. El capítulo *Conclusiones* responde a los objetivos planteados, señala las lecciones aprendidas y traza las líneas de trabajo futuro. El capítulo *Riesgos éticos* examina las consideraciones y salvaguardas asociadas al uso del mecanismo. Cierra el documento un anexo con las instrucciones necesarias para reproducir el experimento completo.

Capítulo 2

Marco de referencia

Este capítulo sienta las bases conceptuales que sostienen el resto del documento. En primer lugar, se delimitan los conceptos centrales —deriva arquitectónica, deuda técnica arquitectónica y *smells*— para, a continuación, presentar las *Fitness Functions* como instrumento de verificación, el análisis estático y los grafos de dependencia como insumo estructural, las métricas de acoplamiento que permiten cuantificarlos y la noción de gobernanza continua como marco de aplicación. Finalmente, se revisan los trabajos previos más relevantes y se contrastan mediante una tabla comparativa, con el fin de evidenciar el vacío que este proyecto pretende ocupar.

2.1. Marco Teórico

2.1.1. Arquitectura de software y atributos de calidad

Antes de abordar la deriva, conviene precisar qué se entiende por arquitectura de software y por qué su integridad importa. La arquitectura es el conjunto de estructuras principales de un sistema, junto con las relaciones que las conectan y los principios que gobiernan su diseño y evolución (Bass et al., 2012). No es un producto tangible, sino una propiedad emergente que se materializa en las dependencias, las capas, los módulos y los contratos que articulan el código. Su papel es doble: por un lado, sirve de vehículo para alcanzar los atributos de calidad deseados; por otro, actúa como marco de comunicación entre quienes construyen y quienes mantienen el sistema (Perry and Wolf, 1992).

La relevancia de la arquitectura para la mantenibilidad ha sido documentada de forma consistente. Cuando la estructura se degrada, los atributos de calidad no decaen todos a la vez ni de manera uniforme: la mantenibilidad y la evolucionabilidad suelen ser los primeros en resentirse, precisamente porque dependen de que las dependencias sean predecibles y de que los cambios puedan acotarse. Un sistema cuya arquitectura se ha erosionado obliga a sus desarrolladores a razonar sobre un territorio que ya no corresponde a ningún mapa compartido, lo que eleva el costo cognitivo y material de cada modificación (Klein et al., 2022). Esta relación entre estructura y costo de cambio es la razón de fondo por la que la vigilancia arquitectónica no puede considerarse un lujo.

2.1.2. Deriva y erosión arquitectónica

La deriva arquitectónica describe el distanciamiento gradual entre la arquitectura concebida para un sistema y la que efectivamente se materializa en su implementación (Martín et al., 2024). Se distingue de la erosión arquitectónica, término que suele reservarse para el deterioro estructural

ya consumado y observable: la deriva es el proceso, la erosión es su huella acumulada. La distinción y sus múltiples denominaciones fueron sistematizadas por de Silva and Balasubramaniam (2012), quienes al revisar las estrategias disponibles para controlar la erosión las agruparon en tres familias —minimizarla mediante disciplina de proceso, prevenirla mediante restricciones impuestas por construcción y repararla una vez detectada— y señalaron que las aproximaciones basadas en verificación automática de conformidad se sitúan entre la primera y la segunda. Ambos conceptos comparten una misma raíz: las decisiones de implementación que, tomadas de forma incremental y bajo presión de entrega, van apartando al código de las restricciones estructurales que deberían gobernarlo (Anthony et al., 2024). La literatura coincide en señalar que este fenómeno no es un accidente puntual, sino una consecuencia casi inevitable de la evolución de los sistemas, lo que convierte su vigilancia en una tarea permanente y no en una actividad esporádica (Klein et al., 2022). Esta lectura es coherente con los trabajos clásicos sobre mantenimiento de la consistencia arquitectónica, que ya advertían del costo de permitir que la implementación y el diseño se separen (Rosik, 2007), y con las leyes de evolución del software, que prevén la degradación estructural como desenlace natural de todo sistema que crece (Lehman, 1980). De ahí que la deriva no deba tratarse como un fallo excepcional del proceso, sino como un riesgo estructural que reclama mecanismos de vigilancia integrados en el propio ciclo de desarrollo.

2.1.3. Deuda técnica arquitectónica y *smells*

El concepto de deuda técnica, acuñado originalmente para describir el costo de las decisiones de diseño postergadas (Cunningham, 1992), trasladado al plano estructural, da lugar a la deuda técnica arquitectónica: el costo diferido que una organización asume al postergar la corrección de las desviaciones que degradan la arquitectura. Los *smells* arquitectónicos —ciclos de dependencia, componentes excesivamente acoplados, violaciones de capas, concentraciones de responsabilidad— son los síntomas observables de esa deuda (Sas and Avgeriou, 2022). La evidencia empírica ha establecido una relación negativa entre la presencia de estos *smells* y la mantenibilidad de los sistemas, lo que refuerza la necesidad de instrumentos que los detecten de forma objetiva (Jolak et al., 2025; Sas et al., 2019). Para las PyMES, cuya capacidad de reacción ante la deuda acumulada es limitada, el reto no es solo detectar los síntomas, sino hacerlo antes de que su corrección resulte prohibitiva (Hamed et al., 2023).

2.1.4. *Fitness Functions*

Una *Fitness Function* es una verificación objetiva y automatizable que expresa, en términos cuantificables, una propiedad que el sistema debe preservar. El término, tomado de la biología evolutiva, fue adoptado por la ingeniería de software para describir pruebas que no verifican una funcionalidad, sino una característica estructural o de calidad de la arquitectura (Paul and Wang, 2019). La formulación de referencia es la de Ford et al. (2017), quienes las sitúan en el centro de las llamadas arquitecturas evolutivas y proponen clasificarlas según varios ejes: por su alcance, distinguiendo las *atómicas* —que evalúan una única dimensión, como una regla de dependencia entre módulos— de las *holísticas*, que evalúan la interacción de varias; y por su cadencia, separando las

que se ejecutan de forma continua en cada integración de las que se activan de manera puntual o desencadenada por un evento. Las tres reglas que este trabajo implementa son, en esa taxonomía, funciones atómicas de ejecución desencadenada, y la Tasa de Conformidad que las agrega constituye una lectura holística construida sobre ellas. A diferencia de las métricas convencionales, que se limitan a describir el estado del sistema, las *Fitness Functions* incorporan un criterio de juicio: producen un veredicto de cumplimiento o incumplimiento sobre una regla declarada. Esta propiedad las hace idóneas para la gobernanza arquitectónica, pues permiten codificar las decisiones de diseño como comprobaciones ejecutables que se revalúan en cada iteración del desarrollo (Continuous Architecture, 2026; Okoone, 2026). Herramientas como *ArchUnit* han popularizado esta idea en el ecosistema Java, al permitir expresar reglas arquitectónicas como pruebas unitarias (TNG Technology Consulting, 2026). En esta línea, la noción de *guardrails* arquitectónicos ha enriquecido la práctica al distinguir entre reglas que bloquean una integración de inmediato y aquellas que solo registran la desviación para su revisión posterior (Rossini, 2022). Autores más recientes subrayan, además, que el valor de las *Fitness Functions* no reside en la comprobación aislada, sino en su capacidad para automatizar decisiones arquitectónicas que, de otro modo, dependerían de la memoria institucional o de la disciplina individual (Niessen, 2026). Este énfasis en la automatización de decisiones conecta las *Fitness Functions* con la práctica de los *Architecture Decision Records* (ADR), que registran de forma explícita las decisiones de diseño para que puedan verificarse a lo largo del tiempo (AOE Technology Radar, 2026).

2.1.5. Análisis estático y grafos de dependencia

El análisis estático consiste en examinar el código fuente —o sus artefactos compilados— sin ejecutar el programa, con el fin de extraer propiedades estructurales. Cuando el resultado de ese examen se organiza como un grafo dirigido, en el que los nodos representan componentes (paquetes, clases o módulos) y las aristas representan dependencias entre ellos, se obtiene una representación compacta y verificable de la arquitectura real del sistema (Manglaras et al., 2026). Este enfoque es particularmente valioso en el contexto de este trabajo porque permite prescindir de la documentación arquitectónica: la estructura se infiere directamente del código, que es la única fuente de verdad siempre disponible. La elección entre analizar el código fuente o el *bytecode* compilado responde a un compromiso práctico: el primero ofrece mayor riqueza semántica, mientras que el segundo simplifica la resolución de dependencias al operar sobre una representación ya consolidada. En el ecosistema Java, herramientas como *ArchUnit* ejemplifican este segundo enfoque, al importar clases compiladas y construir sobre ellas un modelo navegable de dependencias (codecentric AG, 2026). Los modelos de reflexión de Murphy et al. (2001) y los mecanismos de sincronización arquitectónica (Diaz-Pace et al., 2011) constituyen antecedentes directos de esta manera de contrastar el código con las expectativas estructurales.

Representar la estructura como un grafo dirigido no es una comodidad de notación: habilita el uso de algoritmos con propiedades conocidas. La detección de ciclos, en particular, se reduce a hallar los componentes fuertemente conexos del grafo, problema que Tarjan (1972) resolvió con un algoritmo de una sola pasada en profundidad y complejidad lineal en el número de nodos y aristas.

Un componente fuertemente conexo de tamaño mayor que uno es, por definición, un conjunto de paquetes mutuamente alcanzables, es decir, un ciclo. Esta correspondencia entre un concepto arquitectónico y una construcción de la teoría de grafos es la que permite que la verificación sea exacta y no heurística.

2.1.6. Métricas de acoplamiento

Algunas propiedades estructurales no se expresan como una prohibición binaria sino como una cuestión de grado, y para ellas hace falta una medida. Martin (2003) introdujo dos métricas que se han vuelto estándar en el análisis de dependencias entre paquetes: el **acoplamiento eferente** (Ce), que cuenta cuántos paquetes distintos conoce un paquete dado, y el **acoplamiento aferente** (Ca), que cuenta cuántos dependen de él. La primera mide la exposición del paquete a los cambios ajenos; la segunda, su responsabilidad sobre los cambios propios.

Un valor elevado de Ce señala un paquete que ha ido acumulando conocimiento de todo el sistema, y es el indicio habitual de lo que la literatura denomina componente dios: un módulo del que todo depende y que depende de todo, cuya modificación repercute en cascada y cuyo reemplazo se vuelve inviable (Sas and Avgeriou, 2022). Conviene subrayar, no obstante, que la métrica es puramente topológica y no distingue el papel que el paquete desempeña: una raíz de composición o una fachada presentan por diseño un Ce alto sin que ello constituya una deriva. De ahí que el umbral a partir del cual un valor se considera excesivo no sea universal, sino un parámetro que debe calibrarse según el sistema y el rol de cada componente.

2.1.7. Conformidad arquitectónica y gobernanza continua

La conformidad arquitectónica es el grado en que la implementación de un sistema se adhiere a las reglas estructurales que se han definido para él (Martín et al., 2024; Klein et al., 2022). Su verificación no es un fin en sí mismo, sino el insumo de una práctica más amplia: la gobernanza continua, entendida como la vigilancia sistemática y automatizada de las propiedades arquitectónicas a lo largo de todo el ciclo de vida del producto. En organizaciones pequeñas, donde los marcos de control deben ser livianos y proporcionados a los recursos disponibles, la gobernanza continua solo es viable si se apoya en instrumentos de bajo costo y de interpretación sencilla (Modawski and Wolniak, 2025). Esta constatación orienta el diseño del mecanismo que se describe en los capítulos siguientes: un instrumento que agregue las verificaciones individuales en una señal global comprensible para un equipo de desarrollo sin especialistas dedicados. A la vigilancia automatizada se suma, como complemento, la documentación de decisiones arquitectónicas. Los *ADR* proporcionan el registro deliberado de por qué se adoptó una estructura y no otra, y constituyen el insumo natural contra el cual contrastar las desviaciones detectadas por el mecanismo (AOE Technology Radar, 2026). De esta manera, la gobernanza continua no se reduce a un acto de verificación, sino que articula un ciclo de documentación, monitoreo y corrección.

2.2. Estado del Arte

La detección de la deriva arquitectónica ha sido abordada desde múltiples frentes, que conviene examinar para situar la contribución de este proyecto. Un primer grupo de trabajos se concentra en la verificación de conformidad mediante modelos explícitos. Los modelos de reflexión de Murphy et al. (2001) comparan una representación de alto nivel con la estructura extraída del código, pero presuponen la existencia y el mantenimiento de ese modelo de alto nivel, un requisito que las PyMES rara vez satisfacen. Knodel and Popescu (2007) sistematizaron esta familia al comparar los tres enfoques disponibles para la verificación estática de conformidad —modelos de reflexión, matrices de dependencias y lenguajes de reglas— y concluyeron que ninguno domina a los demás en todas las dimensiones: los modelos de reflexión ofrecen mayor poder expresivo a cambio de exigir una arquitectura de referencia, mientras que los lenguajes de reglas resultan más económicos de adoptar pero cubren un espectro menor de restricciones. Esa constatación es directamente pertinente aquí, pues el presente trabajo se sitúa en la tercera familia y asume conscientemente ese compromiso. En la misma línea, Diaz-Pace et al. (2011) exploran la sincronización entre la arquitectura documentada y la implementación, y Martín et al. (2024) estudian la verificación de conformidad en sistemas autoadaptativos; ambos asumen igualmente una especificación arquitectónica de referencia.

Un segundo grupo aborda la deriva a través de sus síntomas. Sas et al. (2019) investigan la evolución de los *smells* de inestabilidad, y Sas and Avgeriou (2022) proponen un índice de deuda técnica arquitectónica basado en aprendizaje de máquina aplicado a la detección de *smells*. Estos trabajos aportan evidencia valiosa sobre el diagnóstico, pero se orientan a la caracterización del deterioro más que a ofrecer un veredicto de conformidad accionable para un equipo de desarrollo cotidiano.

Un tercer grupo lo constituyen las propuestas centradas en las *Fitness Functions* como práctica. Paul and Wang (2019) y la comunidad de arquitectura continua (Continuous Architecture, 2026; Okoone, 2026) formalizan la idea de verificar propiedades arquitectónicas mediante comprobaciones automatizadas, y *ArchUnit* la materializa en el ecosistema Java (TNG Technology Consulting, 2026). Estos trabajos definen el *cómo* verificar, pero no proponen un mecanismo que agregue los resultados en una clasificación global del estado del sistema. Por último, trabajos recientes como Manglaras et al. (2026) reconstruyen arquitecturas de sistemas distribuidos a partir del código, lo que confirma el interés creciente por extraer estructura sin depender de documentación.

Un cuarto grupo, más reciente, agrupa las herramientas y guías orientadas a hacer práctica la verificación en el día a día de los equipos. Los *guardrails* arquitectónicos, difundidos como complemento de las *Fitness Functions* (Rossini, 2022), y las soluciones de verificación por convención para distintos ecosistemas —como *NetArchTest* para .NET (Panneerselvam, 2023) o *ArchUnit* para Java (codecentric AG, 2026)— evidencian que la industria ya demanda instrumentos ligeros de control estructural. Sin embargo, estas herramientas comparten una limitación: exigen que alguien escriba y mantenga las reglas, y no ofrecen por sí solas una lectura agregada del estado global del sistema.

La tabla 2.1 sintetiza esta comparación en torno a cuatro criterios: el enfoque adoptado, si la propuesta detecta derivas de forma automatizada, si prescinde de documentación arquitectónica externa y si resulta asequible para una PyME sin especialistas dedicados.

Tabla 2.1: Comparación de los trabajos relacionados con la propuesta. Leyenda: ✓ lo cumple, ∼ lo cumple parcialmente, × no lo cumple.

Trabajo	**Enfoque**	**Automática**	**Sin doc. externa**	**Veredicto agregado**	**Asequible PyME**
Murphy et al. (2001)	Modelos de reflexión	×	×	×	×
Knodel and Popescu (2007)	Comparación de enfoques de conformidad	✓	×	×	∼
Diaz-Pace et al. (2011)	Sincronización arquitectónica	∼	×	×	×
Martín et al. (2024)	Verificación de conformidad (MAPE-K)	✓	×	×	×
Sas et al. (2019)	Evolución de *smells* de inestabilidad	∼	✓	×	∼
Sas and Avgeriou (2022)	Índice de deuda técnica con aprendizaje de máquina	∼	✓	✓	∼
Ford et al. (2017); Paul and Wang (2019)	*Fitness Functions* como práctica	✓	∼	×	∼
TNG Technology Consulting (2026)	Reglas arquitectónicas como pruebas	✓	∼	×	∼
Manglaras et al. (2026)	Reconstrucción de arquitectura	✓	✓	×	∼
Esta propuesta	*Fitness Functions* sobre grafos, con tasa de conformidad y severidad	✓	✓	✓	✓

La columna «veredicto agregado» merece una precisión, porque es la que sostiene el argumento de la brecha. Solo Sas and Avgeriou (2022) producen una cifra global, pero lo hacen mediante un modelo de aprendizaje de máquina cuyo resultado no es reconstruible por el equipo que lo recibe; el índice indica cuánta deuda hay, no qué reglas se incumplieron ni dónde. La agregación que aquí se propone persigue el objetivo contrario: una cifra deliberadamente simple, calculada como un cociente que cualquier miembro del equipo puede rehacer a mano a partir de los hallazgos que la componen.

El examen anterior revela una brecha consistente: los enfoques basados en modelos exigen documentación que las PyMES no mantienen; los enfoques basados en *smells* describen el deterioro sin ofrecer un veredicto agregado; y las propuestas de *Fitness Functions* aportan el mecanismo de verificación sin resolver la agregación en un diagnóstico clasificatorio. Ninguno de los trabajos revisados combina, en un solo instrumento ligero y sin dependencias de documentación externa, la extracción del grafo de dependencias, la evaluación mediante *Fitness Functions* y la clasificación del estado de conformidad en niveles interpretables. Es precisamente ese espacio el que ocupa la presente investigación.

2.3. Resumen del capítulo

Este capítulo definió los conceptos fundamentales que articulan el proyecto: deriva y erosión arquitectónica, deuda técnica arquitectónica y *smells*, *Fitness Functions*, análisis estático con grafos de dependencia, las métricas de acoplamiento que permiten cuantificar la concentración de responsabilidades y la gobernanza continua como marco de aplicación. Dos correspondencias resultan especialmente relevantes para los capítulos siguientes: la que vincula la noción de ciclo de dependencia con los componentes fuertemente conexos de un grafo, que permite verificarla de forma exacta (Tarjan, 1972), y la que vincula el componente dios con el acoplamiento eferente, que permite medirlo aunque exija calibrar su umbral (Martin, 2003). Sobre esa base, se revisaron los trabajos previos agrupándolos en cuatro corrientes —verificación basada en modelos, detección de *smells*, *Fitness Functions* como práctica y herramientas de verificación por convención— y se los contrastó mediante una tabla comparativa. La comparación evidenció la brecha que motiva la propuesta: la ausencia de un mecanismo que integre, de forma asequible y sin documentación externa, la detección y una clasificación agregada del estado de conformidad que siga siendo reconstruible por quien la recibe.

Capítulo 3

Desarrollo del Proyecto

Este capítulo describe la construcción del mecanismo de detección de deriva arquitectónica, desde su concepción hasta su materialización como prototipo de software. Se presentan la descripción de la propuesta, los requisitos que la delimitan, el diseño de alto nivel, el catálogo de *Fitness Functions* que constituye el núcleo normativo del instrumento y, por último, los detalles de la implementación realizada en Java junto con los escenarios que sirvieron para evaluarla.

3.1. Descripción de la propuesta

El mecanismo propuesto recibe como entrada el código de un sistema Java y produce como salida un diagnóstico sobre su conformidad arquitectónica. Su propósito es ofrecer a los equipos de las PyMES una señal objetiva, reproducible y de bajo costo sobre el estado estructural de sus proyectos, sin exigirles documentación arquitectónica ni conocimientos especializados en verificación. Los usuarios objetivo son, por tanto, los desarrolladores, los líderes técnicos y los arquitectos de organizaciones pequeñas, que necesitan detectar tempranamente las desviaciones antes de que se conviertan en deuda difícil de revertir.

La lógica del instrumento es sencilla y se resume en tres pasos encadenados. Primero, un componente de análisis estático recorre el código compilado y construye un grafo dirigido en el que los nodos son paquetes y las aristas representan dependencias entre ellos. Segundo, un conjunto de tres *Fitness Functions* evalúa sobre ese grafo otras tantas dimensiones: la integridad de las capas, la ausencia de ciclos y el control de la concentración de dependencias salientes. Tercero, un motor de validación agrega los resultados en una Tasa de Conformidad (CR) y clasifica el estado del sistema en uno de tres niveles: *aceptable*, *en observación* o *no conforme*. La figura 3.1 representa esa secuencia y los insumos que la alimentan.

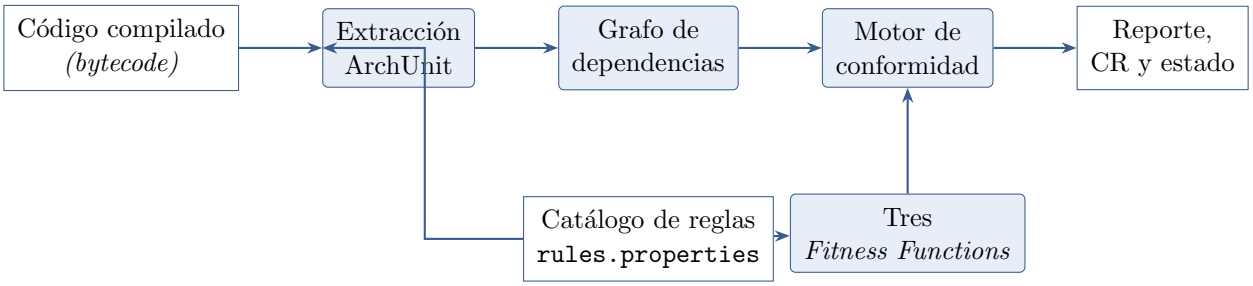


Figura 3.1: Flujo del mecanismo de detección de deriva arquitectónica. El catálogo de reglas es un insumo externo: determina qué se considera admisible sin alterar el código del motor.

3.2. Requisitos

La definición del mecanismo se apoyó en un conjunto de requisitos funcionales y no funcionales que orientaron tanto el diseño como la implementación.

Requisitos funcionales:

- **RF1.** Construir un grafo de dependencias entre paquetes a partir del código compilado, descartando las clases de prueba y las dependencias externas al sistema analizado.
- **RF2.** Verificar la integridad de capas: toda dependencia entre capas debe respetar las adyacencias permitidas que se declaren en la configuración.
- **RF3.** Detectar dependencias cíclicas entre paquetes mediante componentes fuertemente conexos.
- **RF4.** Medir la concentración de dependencias salientes de cada paquete y reportar como violación aquella que supere un umbral configurable.
- **RF5.** Agregar los resultados de las verificaciones en una Tasa de Conformidad y clasificar el estado arquitectónico del sistema, emitiendo un reporte legible.
- **RF6.** Acompañar cada hallazgo de la evidencia estructural que lo origina, hasta el nivel de la referencia concreta entre clases, y declarar en el reporte el alcance del diagnóstico emitido.
- **RF7.** Representar gráficamente el grafo de dependencias, ordenado según la estratificación declarada y con los hallazgos resaltados, en un formato que pueda consultarse sin instalar herramientas adicionales.
- **RF8.** Ofrecer una interfaz gráfica desde la cual seleccionar el proyecto a analizar —la carpeta tal como la deja su herramienta de construcción, o su archivo empaquetado— y consultar el diagnóstico, el grafo y la evidencia sin recurrir a la línea de comandos.

- **RF9.** Asistir, de forma opcional, la redacción del catálogo de reglas de un proyecto que no lo tiene, proponiendo sus parámetros con la justificación de cada decisión y dejando la adopción en manos de la persona.

Requisitos no funcionales:

- **RNF1.** Configurabilidad: las capas, las adyacencias permitidas, las exenciones, los umbrales y el criterio de clasificación deben poder declararse en un archivo externo sin modificar el código, y toda exención aplicada debe quedar registrada en el reporte.
- **RNF2.** Ligereza: el mecanismo debe poder ejecutarse en un entorno de desarrollo convencional, sin infraestructura adicional ni dependencias de servicios externos.
- **RNF3.** Interpretabilidad: el reporte resultante debe ser comprensible para un profesional sin formación específica en verificación arquitectónica.
- **RNF4.** Reproducibilidad: ante la misma entrada, el mecanismo debe producir el mismo diagnóstico, y debe poder emitirlo en un formato verificable por otras herramientas.

El requisito RF6 no figuraba en la formulación inicial: se incorporó durante la evaluación, al constatar que la capacidad explicativa era el único indicador con margen de mejora. Su origen se documenta en el capítulo siguiente, pues ilustra el carácter iterativo que la metodología de diseño prescribe (Peffers et al., 2007). Tampoco figuraban RF7, RF8 y RF9, que responden a una misma constatación posterior: un diagnóstico correcto aporta poco si quien debe leerlo necesita, para obtenerlo, conocer la línea de comandos, la disposición interna de su herramienta de construcción o el formato del catálogo. Los tres sirven a RNF3 sin alterar el núcleo del mecanismo.

3.3. Diseño de la propuesta

El diseño se organizó en torno a una arquitectura de módulos con responsabilidades separadas, de modo que cada etapa del proceso pudiera desarrollarse y verificarse de forma independiente. La figura 3.2 muestra los cinco módulos del núcleo, los que lo ponen al alcance del usuario —un servicio de análisis, una interfaz gráfica y un asistente opcional—, la raíz de composición que los ensambla y el sentido en el que fluyen sus dependencias.

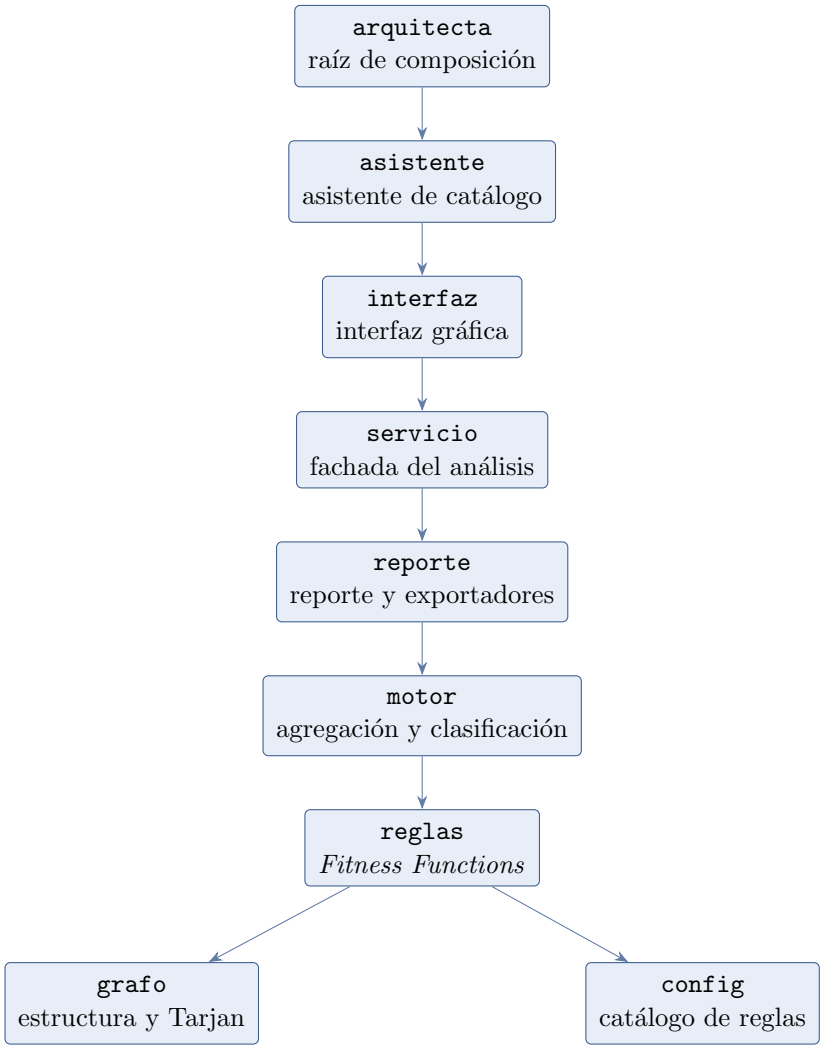


Figura 3.2: Módulos del prototipo y sentido de sus dependencias. Se omiten, por legibilidad, las aristas que saltan niveles —de la raíz a la interfaz y al servicio, del asistente al servicio, de la interfaz al reporte o hacia `grafo` y `config`—; ningún módulo depende de otro situado por encima de él, de modo que el grafo interno es acíclico. La interfaz, además, no accede a `grafo` ni a `config`: solo presenta lo que el servicio le entrega.

El módulo de *configuración* carga y representa el catálogo de reglas (raíz de paquetes, capas, adyacencias permitidas, umbrales y modo de clasificación). El módulo de *grafo* construye y almacena la estructura de dependencias, calcula los componentes fuertemente conexos mediante el algoritmo de Tarjan (Tarjan, 1972) y reconstruye la ruta concreta de cada ciclo. El módulo de *reglas* encapsula las tres *Fitness Functions*, cada una de las cuales define sus criterios de verificación y de incumplimiento. El módulo *motor* agrega los resultados y produce la clasificación. El módulo de *reporte* traduce el diagnóstico a un formato legible y a formatos verificables por otras herramientas.

Por encima del núcleo, el módulo de *servicio* actúa como fachada: localiza el código compilado del proyecto elegido, carga su catálogo y encadena las etapas anteriores, de modo que la línea de comandos y la interfaz gráfica comparten un único camino de análisis y no pueden discrepar. El módulo de *interfaz* presenta ese resultado de forma interactiva. Por último, el módulo *asistente* ayuda a redactar el catálogo con un modelo de lenguaje; se acopla a la interfaz a través de un punto de extensión, de modo que ni la interfaz ni el núcleo dependen de él.

3.4. Catálogo de *Fitness Functions*

El primer objetivo específico exigía formular un catálogo acotado de *Fitness Functions* en el que cada regla dejara explícita su racionalidad, la evidencia que espera encontrar y el criterio con el que declara un incumplimiento. La tabla 3.1 recoge ese catálogo, que constituye el núcleo normativo del mecanismo: define, en términos verificables, qué se considera una arquitectura sana dentro del alcance del proyecto.

Este catálogo no vive únicamente en el documento. Cada regla se autodescribe en el código mediante los métodos que declaran su propiedad, su racionalidad, su unidad de verificación, su criterio de incumplimiento y su severidad, y el prototipo puede imprimirlo bajo demanda con la opción `--catalogo`. La decisión persigue un fin concreto: impedir que el catálogo publicado y el efectivamente aplicado diverjan con el tiempo, que es precisamente la clase de deriva que el proyecto combate.

3.5. Operacionalización de la Tasa de Conformidad

La Tasa de Conformidad es la pieza que convierte un conjunto de hallazgos dispersos en una señal única, y por ello merece una justificación explícita de cómo se calcula. El anteproyecto la definió como la proporción de reglas cumplidas frente al total de reglas evaluadas. Con un catálogo de tres reglas, esa formulación admite únicamente cuatro valores posibles —0 %, 33 %, 67 % y 100 %—, lo que la vuelve incapaz de distinguir entre un sistema con una violación de capas y otro con veinte. Se optó, por tanto, por conservar la definición como cociente pero desplazando la unidad de conteo desde la regla hasta la **verificación atómica**: cada arista entre capas distintas, en el caso de FF1, y cada paquete, en el de FF2 y FF3. Formalmente,

$$CR = 100 \times \frac{V - I}{V}, \tag{3.1}$$

donde V es el número total de verificaciones atómicas realizadas por las tres reglas e I el número de las que fallaron. Cuando $V = 0$ —un sistema sin paquetes analizables— la CR se define como 100 % por convención, pues no existe evidencia de incumplimiento.

Este refinamiento es una precisión de la definición aprobada, no una sustitución: la CR sigue siendo una proporción explícita de cumplimiento, sigue interpretándose con los mismos umbrales del 85 % y el 70 %, y sigue siendo reconstruible a partir de los hallazgos individuales. Lo que gana

Tabla 3.1: Catálogo de *Fitness Functions* implementadas.

	FF1 — Integridad de capas	**FF2 — Ausencia de ciclos**	**FF3 — Concentración de dependencias**
Propiedad protegida	La estratificación declarada se respeta.	El grafo de dependencias entre paquetes es acíclico.	Ningún paquete concentra más dependencias salientes de las admisibles.
Racionalidad	Una dependencia que salta o invierte capas anula la separación de responsabilidades y propaga los cambios fuera de su ámbito previsto.	Los paquetes de un ciclo no pueden compilarse, probarse ni comprenderse por separado: forman una unidad de cambio que ninguna decisión de diseño declaró.	Un acoplamiento eferente elevado señala un paquete que asumió más responsabilidades de las que le corresponden; su modificación arrastra al sistema entero (Martin, 2003).
Unidad de verificación	Cada arista entre dos capas declaradas distintas.	Cada paquete del grafo.	Cada paquete del grafo.
Criterio de incumplimiento	El par (capa origen, capa destino) no figura en la matriz de adyacencias permitidas.	El paquete pertenece a un componente fuertemente conexo de tamaño mayor que uno.	El acoplamiento eferente Ce del paquete supera el umbral configurado.
Evidencia esperada	Arista infractora, par de capas, adyacencias autorizadas y referencias entre clases que la originan.	Ruta cíclica concreta, paquetes del componente y referencias entre clases que cierran el ciclo.	Valores de Ce y Ca, umbral aplicado y lista de paquetes destino.
Severidad	Mayor	Crítica	Mayor
Parámetro	Matriz `permitido.*`	Ninguno	`umbral.concentracion`

es resolución, y con ella la posibilidad de que la métrica refleje la magnitud del deterioro y no solo su presencia.

Conviene, no obstante, dejar constancia de las tres limitaciones que la fórmula arrastra, porque condicionan cómo debe leerse cualquier valor que produzca.

- **Las unidades son heterogéneas.** FF1 cuenta aristas mientras FF2 y FF3 cuentan paquetes, de modo que el peso relativo de cada regla en la CR depende de la topología del sistema. Un sistema con muchas dependencias entre capas y pocos paquetes concede más peso a FF1; uno con muchos paquetes y pocas aristas inter-capa, a FF2 y FF3.
- **El denominador no es constante entre versiones.** Introducir un paquete nuevo aumenta V aunque no introduzca ninguna violación. Ello implica que la CR de dos versiones de un sistema es comparable solo si se acompaña del conteo de verificaciones, razón por la cual el reporte lo declara siempre.

- **La proporción diluye las violaciones localizadas.** A medida que crece el sistema, una violación aislada representa una fracción cada vez menor de V. En un sistema de quinientos paquetes, un ciclo entre dos de ellos apenas movería la CR. Esta propiedad no es un defecto de implementación sino una consecuencia matemática de toda métrica proporcional, y su alcance real se midió durante la evaluación.

La tercera limitación resultó ser la más relevante y motivó dos respuestas complementarias. La primera afecta a la clasificación y se describe en el capítulo de evaluación, una vez presentada la evidencia que la justifica. La segunda afecta a la lectura misma de la métrica y conviene enunciarla aquí, porque se desprende directamente de la fórmula.

Si el valor absoluto de la CR se diluye con el tamaño del sistema, su **variación** no lo hace. Un descenso de cuatro puntos entre dos corridas es igual de significativo en un sistema de cinco paquetes que en uno de quinientos, porque se mide contra el propio sistema y no contra un umbral universal: el denominador, que es la fuente de la dilución, es prácticamente el mismo en ambas mediciones. El mecanismo registra por ello cada corrida y, cuando dispone de historia previa para el mismo sistema, acompaña la cifra con su variación en puntos porcentuales:

```
Tasa de Conformidad     : 86.41%
Tendencia               : +4.10 pp respecto de la corrida
                          anterior (82.31%) - mejora
```

La distinción entre nivel y tendencia no es un adorno. Un sistema grande puede incumplir de forma persistente sin que su CR baje nunca del umbral de aceptación, y en ese régimen el valor absoluto no informa; la serie, en cambio, revela si el deterioro avanza o se contiene. Es la diferencia entre un detector puntual y un indicador prospectivo, y es la forma en que la métrica resulta útil en el escenario de gobernanza continua que motiva el proyecto.

3.6. Decisiones de diseño

La materialización del mecanismo exigió resolver una serie de decisiones que, aunque técnicas, tienen consecuencias conceptuales.

La primera fue la elección del nivel de granularidad: se optó por construir el grafo a nivel de paquetes y no de clases. Esta decisión sacrifica algo de precisión en favor de dos beneficios: reduce el volumen de aristas, lo que hace el grafo más legible y el diagnóstico más comprensible, y amortigua el ruido que introducen las dependencias internas entre clases de un mismo paquete, que carecen de relevancia arquitectónica.

La segunda fue la elección de la fuente del análisis: se prefirió el *bytecode* compilado al código fuente, porque permite resolver las dependencias de forma completa —incluidas las referencias implícitas que el compilador materializa— sin ejecutar el sistema (codecentric AG, 2026). Esta preferencia coincide con la de la mayoría de las herramientas de verificación de conformidad revisadas en su momento por Knodel and Popescu (2007).

La tercera, de mayor calado, fue la selección de las tres dimensiones evaluadas. La integridad de capas captura la violación más frecuente y más fácil de comunicar; la ausencia de ciclos ataca una de las estructuras más dañinas para la mantenibilidad, pues un ciclo obliga a modificar varios módulos a la vez; y la concentración de dependencias salientes detecta los componentes que, sin proponérselo, se convierten en el centro gravitacional del sistema. Estas tres dimensiones no agotan el espectro de la deriva, pero ofrecen una cobertura inicial suficiente para demostrar la viabilidad del enfoque, y su elección deliberada mantiene el catálogo acotado, en consonancia con la búsqueda de un instrumento ligero.

La cuarta fue la separación estricta entre la declaración de las reglas y su evaluación. Al externalizar la configuración a un archivo de propiedades, se garantiza que el mismo motor pueda reutilizarse sobre proyectos con arquitecturas distintas sin modificar una sola línea de código, un requisito de configurabilidad que se consideró irrenunciable para la adopción en PyMES. Esta decisión, además, convierte al mecanismo en un instrumento genérico: el conocimiento sobre qué es admisible y qué no reside en la configuración, no en el código.

La quinta afectó al contrato de las reglas. Una primera versión separaba la evaluación en tres operaciones independientes —obtener los hallazgos, contar las verificaciones y contar los incumplimientos—, lo que abría la puerta a que la cifra que alimenta la CR y la evidencia que la explica procedieran de recorridos distintos y llegaran a discrepar. El contrato definitivo obliga a cada regla a resolver todo en una única pasada y a devolver conteos y hallazgos en un mismo objeto, cuya construcción rechaza combinaciones imposibles. La coherencia entre el número y su justificación deja así de depender de la disciplina del programador y pasa a estar garantizada por el diseño.

La sexta consistió en conservar, junto a cada arista del grafo, la evidencia que la originó. Un diagnóstico que afirma que «persistencia depende de presentación» obliga al equipo a buscar a mano dónde ocurre eso; uno que señala el método y la línea exactos convierte el hallazgo en una tarea accionable. El costo en memoria es proporcional al número de referencias del sistema y resultó despreciable en los escenarios evaluados.

3.7. Implementación de la propuesta

La implementación se desarrolló en Java, con nivel de lenguaje 17, y se organizó como un proyecto Maven bajo el identificador `co.javeriana.msw:arquitecta`. Para la construcción del grafo se empleó la biblioteca *ArchUnit* en su versión 1.3.0, que proporciona importadores de *bytecode* capaces de extraer las dependencias entre clases sin ejecutar el sistema (TNG Technology Consulting, 2026); el resto de la lógica —incluidos el algoritmo de detección de ciclos, el motor y los exportadores— se implementó sin dependencias externas, con el fin de mantener el mecanismo ligero y comprensible, en coherencia con RNF2.

El código se estructuró en los paquetes `co.javeriana.msw.arquitecta.config`, `.grafo`, `.reglas`, `.motor`, `.reporte`, `.servicio`, `.interfaz` y `.asistente`, bajo una raíz de composición que los ensambla. En el paquete de configuración, las clases `Capa` y `ConfiguracionReglas` modelan respectivamente cada capa arquitectónica y el catálogo completo, mientras que

`CargadorConfiguracion` interpreta un archivo de propiedades y valida su consistencia —rechaza capas declaradas sin paquetes, adyacencias hacia capas inexistentes, umbrales no numéricos y umbrales invertidos— antes de que el análisis comience. El formato es el siguiente:

```
raiz=com.caso
capas=presentacion,negocio,persistencia
capa.presentacion=com.caso.presentacion
capa.negocio=com.caso.negocio
capa.persistencia=com.caso.persistencia
permitido.presentacion=negocio
permitido.negocio=persistencia
permitido.persistencia=
excluir=
rol.compartido=
umbral.concentracion=3
umbral.cr.aceptable=85
umbral.cr.observacion=70
clasificacion.modo=cr+severidad
```

La asignación de paquetes a capas se resuelve por prefijo, de modo que los subpaquetes heredan la capa de su padre. Los paquetes que no encajan en ningún prefijo declarado —un dominio compartido, por ejemplo— quedan deliberadamente fuera de FF1: al no pertenecer a la estratificación, no puede afirmarse que la vulneren.

3.7.1. Exenciones declaradas

Las dos últimas propiedades del ejemplo merecen una explicación aparte, porque responden a un problema que aparece en cuanto el mecanismo se aplica a un sistema que no fue construido para él. Un catálogo que no admita excepciones obliga a elegir entre dos salidas igualmente malas: tolerar falsos positivos hasta que el equipo deje de leer el reporte, o desactivar la regla entera y perder también los hallazgos legítimos. La configuración distingue por ello dos situaciones que, aunque se parecen, son de naturaleza distinta.

La propiedad `excluir` señala código que **no forma parte del sistema evaluado** aunque el empaquetado lo sitúe bajo la misma raíz: dependencias vendorizadas, código generado, módulos de terceros embebidos. Esos paquetes se retiran del grafo por completo. La exclusión alcanza al paquete y a todos sus subpaquetes, porque el código ajeno siempre llega en forma de subárbol.

La propiedad `rol.compartido`, en cambio, señala paquetes que **sí pertenecen al sistema** pero cuyo papel arquitectónico —un núcleo de anotaciones, una raíz de composición, una fachada— hace que concentren dependencias por diseño y no por deterioro. Estos paquetes permanecen en el grafo y en la evidencia, pero quedan exentos de FF2 y FF3. Aquí la coincidencia es exacta y no se hereda a los subpaquetes, y la razón es conceptual: un papel lo desempeña un paquete concreto, no un

subárbol. Propagarlo hacia abajo eximiría al sistema entero cada vez que el núcleo compartido resultara ser el paquete raíz, que es precisamente el caso más frecuente.

Conviene subrayar qué clase de mecanismo es este. No se trata de una vía para silenciar hallazgos incómodos, sino de la forma en que el anteproyecto había previsto contener el riesgo de formalismo vacío: mediante el *registro de excepciones justificadas*. De ahí que la exención no viva en el código de la regla sino en la configuración, donde es visible y discutible, y que el reporte enumere en su sección de alcance todas las exenciones aplicadas junto con las reglas que cada una afecta. Una excepción declarada y registrada puede auditarse; una codificada dentro de la regla, no.

En el paquete de grafo, `GrafoDependencias` representa el grafo dirigido, descarta los autobucles y conserva la evidencia de cada arista; `ConstructorGrafo` lo puebla a partir del *bytecode* mediante *ArchUnit*, restringiendo el análisis al paquete raíz para que las bibliotecas de terceros no contaminen el diagnóstico; `Tarjan` calcula los componentes fuertemente conexos; y `BuscadorCiclos` reconstruye, dentro de cada componente, el recorrido concreto que cierra el ciclo. En el paquete de reglas se definieron la interfaz `Regla` y sus tres implementaciones, `ReglaIntegridadCapas`, `ReglaCiclos` y `ReglaConcentracion`, que materializan las *Fitness Functions* del catálogo y satisfacen los requisitos RF2, RF3 y RF4. Cada hallazgo se modela mediante la clase `Violacion`, que distingue la *ubicación* —los paquetes implicados, que responden a la pregunta «dónde»— de la *evidencia* —las referencias entre clases y los valores medidos, que responden a «por qué»—, y le asocia una severidad.

El motor de validación, implementado en `MotorConformidad`, aplica la ecuación 3.1 y clasifica el resultado. Los estados posibles se codificaron en el enumerado `EstadoArquitectonico`, declarado de mejor a peor para que el orden natural permita adoptar el veredicto más restrictivo cuando dos criterios discrepan. El resultado agregado se encapsula en la clase `Resultado`, que conserva por separado el estado deducido de la CR y el deducido de la severidad, de modo que la procedencia de la clasificación sea siempre auditable. Finalmente, el paquete de reporte convierte ese resultado en un texto estructurado, en dos formatos verificables por otras herramientas —JSON y CSV— y en una representación gráfica del grafo que se describe en la sección 3.8; la clase `Analisis`, del paquete de servicio, encadena el proceso completo, y `ArquitectaApp` lo expone desde la línea de comandos o, si se invoca sin argumentos, abre la interfaz gráfica descrita en la sección 3.9. El sistema a analizar puede indicarse como un directorio de clases compiladas, como la carpeta raíz de un proyecto o directamente como un archivo `.jar`, que es como suele distribuirse. Si el análisis no encuentra ninguna clase bajo la raíz declarada, el mecanismo termina con un error en lugar de emitir un diagnóstico: un grafo vacío obtendría una CR del 100 % y sería clasificado como aceptable, de modo que apuntar por descuido al directorio de fuentes en lugar del de clases produciría un falso negativo silencioso.

El procedimiento de agregación y clasificación se formaliza en el algoritmo 1. Su diseño responde a un principio de transparencia: la Tasa de Conformidad no se calcula como una cifra opaca, sino como el cociente explícito entre las verificaciones superadas y el total, de modo que cualquier resultado pueda reconstruirse a partir de las evidencias individuales que lo sustentan.

Algorithm 1 Cálculo de la Tasa de Conformidad y clasificación del estado.

$V \leftarrow 0$; $I \leftarrow 0$; $s_{max} \leftarrow \varnothing$
for cada regla r del catálogo **do**
  $(v, i, H) \leftarrow$ evaluar(r, grafo) {una sola pasada por regla}
  $V \leftarrow V + v$; $I \leftarrow I + i$
  for cada hallazgo $h \in H$ **do**
    registrar(h); $s_{max} \leftarrow$ masGrave(s_{max}, severidad(h))
$CR \leftarrow 100 \times (V - I)/V$ {100 si $V = 0$}
$e_{CR} \leftarrow$ Aceptable si $CR \geq 85$; En observación si $CR \geq 70$; No conforme en otro caso
$e_{sev} \leftarrow$ No conforme si s_{max} es crítica; En observación si es mayor; Aceptable si no hubo hallazgos
if modo $=$ *cr* **then**
  estado $\leftarrow e_{CR}$
else
  estado $\leftarrow$ masRestrictivo(e_{CR}, e_{sev})
return Resultado(CR, e_{CR}, s_{max}, e_{sev}, estado, hallazgos)

3.8. Representación gráfica del grafo

Un diagnóstico de conformidad se comprende mucho mejor cuando puede verse. Una lista de violaciones dice qué falla; un dibujo del grafo muestra además dónde está cada fallo respecto de la arquitectura que se pretendía, y permite advertir patrones —un paquete del que todo depende, una capa que concentra los ciclos— que ninguna tabla hace evidentes. El mecanismo dibuja por ello el grafo que extrae, y lo hace sin delegar en herramientas externas.

La decisión de no delegar tiene una razón que va más allá de la ligereza. Los algoritmos genéricos de dibujo de grafos dirigidos, como el que implementa Graphviz (Gansner et al., 1993), colocan los nodos según el sentido de las aristas: un nodo se sitúa por encima de aquellos de los que depende. Aplicado a una arquitectura con deriva, ese criterio produce un resultado engañoso, porque precisamente las violaciones —dependencias que remontan la estratificación— empujan a las capas inferiores hacia arriba y pueden invertir el orden que el dibujo debería mostrar. En un grafo de conformidad, el orden vertical no debe deducirse de las aristas sino de la arquitectura declarada; y cualquier arista que lo contradiga debe dibujarse como lo que es.

El procedimiento sigue el esquema clásico del dibujo de grafos jerárquicos (Sugiyama et al., 1981), adaptado al dominio en cuatro pasos:

1. **Asignación de bandas.** Cada capa declarada ocupa una banda horizontal. Su posición se deduce de la matriz de dependencias permitidas y no del orden en que las capas se escribieron en la configuración: una capa que puede depender de otras se sitúa por encima de ellas. En el caso de *ArchUnit*, cuyas capas se declararon de abajo arriba, el dibujo las muestra igualmente en el orden correcto.

2. **Asignación de filas.** Dentro de cada banda, los paquetes se distribuyen en filas según el

camino más largo sobre la condensación del subgrafo de la banda, de modo que también las dependencias internas apunten hacia abajo. Los paquetes de un mismo ciclo comparten fila, y las filas demasiado anchas se parten para conservar la legibilidad.

3. **Reducción de cruces.** Cada fila se reordena según el baricentro horizontal de sus vecinos, alternando barridos descendentes y ascendentes.

4. **Columna lateral.** Los paquetes sin capa —un dominio compartido, un núcleo de anotaciones— no pertenecen a la estratificación y se colocan en una columna aparte, a la altura media de sus vecinos.

El trazado de las aristas codifica su significado. Las que descienden van del borde inferior del origen al superior del destino; las que remontan la estratificación rodean los nodos por fuera, de modo que se distinguen a simple vista; las que unen paquetes de una misma fila se dibujan como arcos, por arriba en un sentido y por abajo en el otro, para que una dependencia mutua no se superponga consigo misma. Los extremos de las aristas que entran o salen por un mismo lado de un nodo se reparten a lo largo de ese lado, y los paquetes de los que dependen muchos otros se ensanchan en proporción, de modo que las puntas de flecha no se amontonan y el tamaño mismo del recuadro delata a los concentradores. El color y el trazo completan la lectura: rojo continuo para las violaciones de capas, rojo discontinuo para las aristas de un ciclo, ámbar para la concentración excesiva y gris atenuado para lo que concierne a un núcleo compartido exento.

La misma disposición se emite en tres formatos. El **SVG** es una imagen autocontenida que cualquier navegador abre sin instalar nada; cada paquete incluye un texto emergente con su nombre completo, su capa, sus acoplamientos y su estado, de modo que el dibujo puede mantenerse compacto sin perder información. El **TikZ** es la misma figura expresada como código LaTeX, lista para incluirse en un documento. El **DOT** se conserva por interoperabilidad con Graphviz, pero fija las filas calculadas y declara las aristas ascendentes como aristas que no restringen la posición, de modo que tampoco Graphviz pueda invertir las capas.

El formato TikZ merece un comentario adicional, porque afecta a este mismo documento. Las figuras de grafos que aparecen en él —las de los escenarios de evaluación y la del sistema externo analizado en el capítulo siguiente— no se dibujaron a mano: las generó el propio mecanismo a partir del grafo que extrajo. Una figura dibujada a mano puede discrepar del resultado real sin que nada lo delate; una generada por el mecanismo es, por construcción, una representación fiel de lo que este calculó, y puede regenerarse cada vez que el sistema cambia.

3.9. Interfaz gráfica

Los formatos anteriores sirven para documentar y para automatizar, pero presuponen a alguien dispuesto a componer una invocación desde la consola, que sepa además en qué directorio deja su herramienta de construcción las clases compiladas. Para el perfil de PyME que motivó el trabajo esa barrera no es menor: el diagnóstico más útil es el que el líder técnico puede obtener por sí mismo,

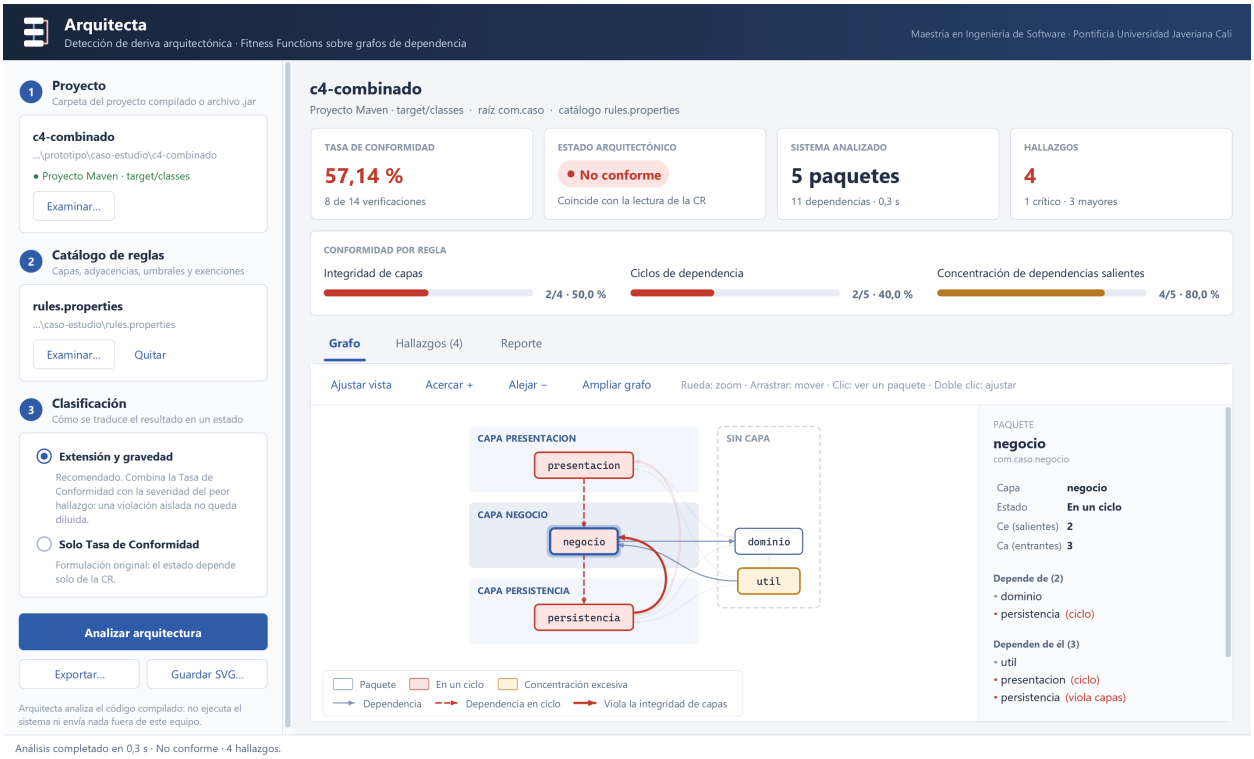


Figura 3.3: Interfaz gráfica del prototipo analizando el escenario combinado (C4), descrito en la sección 3.12. La barra lateral guía los tres pasos —proyecto, catálogo y criterio de clasificación—; el panel principal muestra los indicadores, la conformidad por regla y el grafo, en el que se ha seleccionado el paquete `negocio`: lo que no le concierne se atenúa y la ficha de la derecha detalla su capa, sus acoplamientos, sus vecinos y los hallazgos en que interviene.

sin intermediarios. El prototipo incorpora por ello una interfaz gráfica de escritorio (figura 3.3) que ofrece la misma funcionalidad sin esa barrera, en respuesta a RF8.

El análisis se organiza en tres pasos. En el primero, el usuario elige la **carpeta del proyecto** con un selector o arrastrándola a la ventana. No necesita localizar el directorio de clases: la aplicación reconoce las convenciones de Maven (`target/classes`), Gradle (`build/classes/java/main`), IntelliJ IDEA (`out/production`) y Eclipse (`bin`), así como el `.jar` generado por el proyecto o elegido directamente. Si la carpeta solo contiene fuentes, la interfaz lo advierte y pide compilar, por la misma razón por la que la línea de comandos rechaza un grafo vacío: un análisis sin clases sería un falso negativo. En el segundo paso se fija el **catálogo de reglas**; si el proyecto trae el suyo, se carga sin intervención. Si no lo trae, se aplica una **configuración mínima**: la raíz de análisis se infiere como el prefijo de paquete común a todas las clases y, a falta de capas declaradas, se evalúan solo FF2 y FF3. La interfaz lo señala de forma explícita, porque la estratificación no puede verificarse sin declararla; la configuración mínima es un punto de partida para un primer diagnóstico, no un

sustituto del catálogo. El tercer paso elige el **criterio de clasificación**, con el combinado de la sección 3.5 como opción recomendada.

El resultado se presenta en dos niveles. Arriba, cuatro indicadores —la Tasa de Conformidad, el estado, el tamaño del sistema y los hallazgos— y la conformidad local de cada regla. La CR se acompaña de su variación respecto del análisis anterior del mismo proyecto, que la aplicación conserva entre sesiones, y el estado indica si la severidad endureció el veredicto que la CR sola habría emitido; ambas lecturas responden a las limitaciones de la formulación proporcional discutidas en la sección 3.5. Debajo, tres pestañas. La del **grafo** dibuja la disposición de la sección 3.8 de forma interactiva: admite acercar, desplazar y seleccionar un paquete, lo que atenúa todo lo que no le concierne. La de **hallazgos** los tabula por severidad y, al elegir uno, muestra su evidencia —las referencias concretas entre clases— y lo resalta en el grafo. La del **reporte** reproduce, sin cambios, el reporte textual de la línea de comandos. Desde la misma ventana puede exportarse la evidencia completa en los formatos descritos en el anexo A.

La interfaz no reimplementa ninguna parte del análisis: pide el diagnóstico al servicio y dibuja lo que este le devuelve. De ahí se siguen dos garantías. La primera es la equivalencia de resultados, que la batería de pruebas comprueba de forma explícita: un mismo sistema obtiene idéntico diagnóstico analizado como directorio de clases, como carpeta de un proyecto Maven o como `.jar`, y cualquiera de esas vías reproduce las cifras del experimento. La segunda es que la separación no depende de la disciplina de quien mantenga el código, porque el catálogo del autoanálisis prohíbe a la interfaz acceder directamente a la configuración y a la extracción; si alguna vez lo hiciera, el propio mecanismo lo señalaría como deriva. La interfaz se construyó con Swing, la biblioteca gráfica incluida en el JDK, de modo que tampoco añade dependencias y RNF2 se mantiene. La aplicación se abre con doble clic sobre el lanzador del prototipo, que lo compila la primera vez y cada vez que su código cambia, y su pantalla de inicio ofrece los cinco escenarios del caso de estudio para analizarlos con un clic: quien quiera contrastar los diagnósticos del capítulo siguiente puede hacerlo sin preparar nada.

3.10. Asistente para la definición del catálogo

Para un proyecto sin catálogo, la interfaz aplica la configuración mínima, que no puede verificar la estratificación: FF2 no necesita parámetros y FF3 tiene un umbral por defecto, pero FF1 exige declarar qué capas existen y quién puede depender de quién. Esa declaración no puede inferirse del código sin desnaturalizar el diagnóstico. La deriva es precisamente la diferencia entre la arquitectura prevista y la implementada; un catálogo deducido de la implementación tomaría como referencia la arquitectura que el código tiene hoy, legitimaría la deriva acumulada y garantizaría que nunca se detectara. Es el mismo principio de los modelos de reflexión de Murphy et al. (2001): el código se contrasta con un modelo que declaran las personas. Tampoco existe un estándar del que derivar las reglas de un sistema concreto: Ford et al. (2017) presentan las *Fitness Functions* como verificaciones que cada equipo define para las propiedades que le importan.

Lo que sí puede hacerse es abaratar esa declaración. El prototipo incorpora con ese fin un asistente opcional (figura 3.4) que conversa con un modelo de lenguaje para proponer el catálogo,

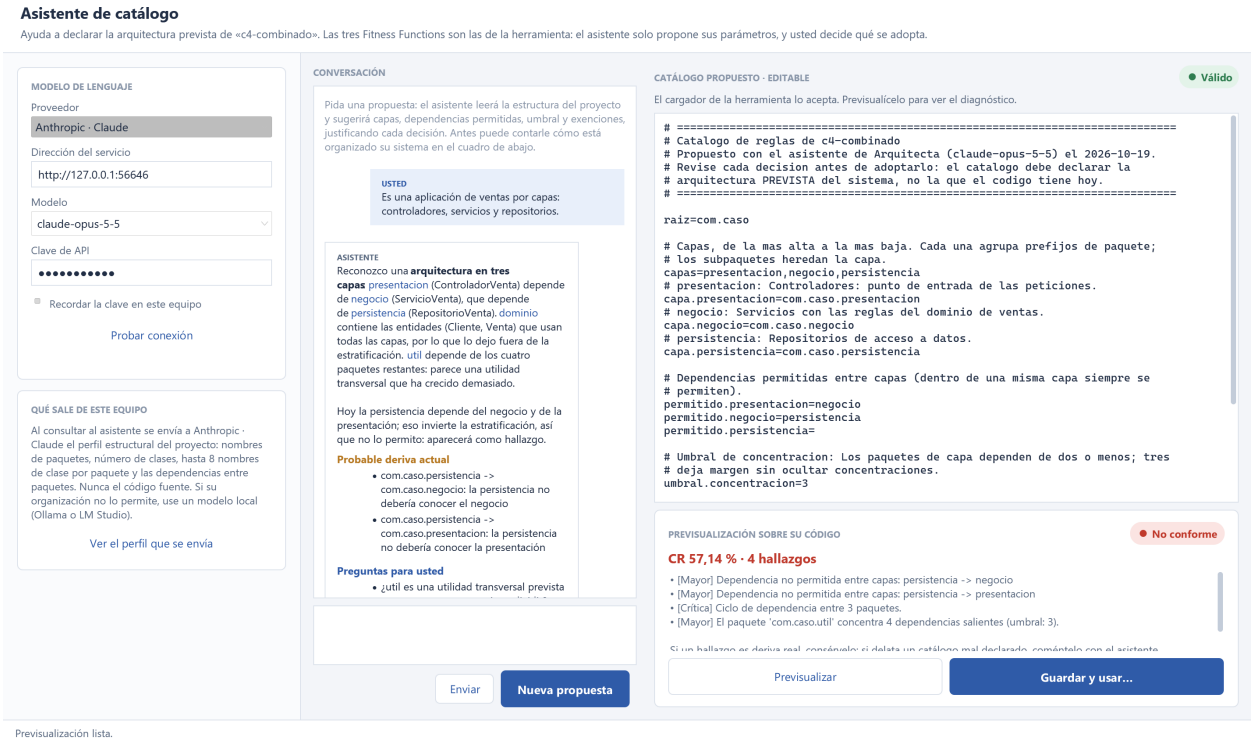


Figura 3.4: Asistente de catálogo sobre el escenario C4. A la izquierda, la configuración del modelo y lo que se envía; en el centro, la conversación, con la deriva probable y las preguntas para la persona; a la derecha, el catálogo propuesto, editable y validado, y su previsualización sobre el código, que reproduce el diagnóstico de la tabla 4.2. La respuesta del modelo se simuló para la captura.

en respuesta a RF9. Su diseño parte de tres restricciones. La primera es que **no define reglas**: las tres *Fitness Functions* son las de la herramienta, y el asistente solo propone sus parámetros —capas y paquetes, dependencias permitidas, umbral de concentración y exenciones—, justificando cada decisión y formulando como preguntas lo que no puede saber. La segunda es que **no legitima la deriva**: sus instrucciones le exigen declarar la arquitectura prevista, de modo que una dependencia existente que contradiga la estratificación plausible no se permite, sino que se señala como deriva probable. La tercera es que **no decide**: la propuesta llega a un editor donde la persona la corrige, se valida con el mismo cargador que un catálogo escrito a mano y se previsualiza sobre el código antes de adoptarse. Cada justificación queda como comentario junto a la decisión que explica, de modo que el catálogo resultante es también el registro de por qué se declaró así, en la línea de los *Architecture Decision Records* (AOE Technology Radar, 2026).

El intercambio con el modelo se apoya en un **perfil estructural** del proyecto: los nombres de los paquetes, el número de clases de cada uno, hasta ocho nombres de clase por paquete —que suelen delatar su papel— y las dependencias entre paquetes. Nunca se envía el código fuente, y la

persona puede ver el texto exacto antes de enviarlo. El asistente admite el servicio de Anthropic, el de OpenAI y cualquier servicio compatible con este último, lo que incluye modelos que se ejecutan en el propio equipo mediante Ollama o LM Studio; con estos, nada sale del equipo. La configuración se gestiona desde la misma ventana, y la clave de acceso solo se conserva si la persona lo pide. El cliente usa el soporte HTTP del propio JDK, de modo que RNF2 se mantiene.

La separación de responsabilidades se verifica con el propio mecanismo. El asistente depende de la interfaz —cuyos componentes visuales reutiliza— y del servicio, pero ninguno de los dos depende de él: la ventana principal expone un punto de extensión y es la raíz de composición la que decide ofrecerlo. El catálogo del autoanálisis lo declara así, de modo que una dependencia del núcleo hacia el asistente, o hacia cualquier modelo de lenguaje, aparecería como deriva. Conviene, por último, delimitar su estatus: el asistente es una ayuda a la adopción y **no forma parte del mecanismo evaluado**. Sus pruebas verifican lo que es responsabilidad de la herramienta —qué se envía, cómo se interpreta la respuesta y que la propuesta pase por el mismo cargador y produzca el diagnóstico esperado— frente a un servidor local que imita las API de los modelos; la calidad de las propuestas de un modelo real no se evaluó, y se plantea como trabajo futuro.

3.11. Verificación del prototipo

La construcción se acompañó de dos actividades de verificación que conviene distinguir de la evaluación experimental: aquí no se juzga si el mecanismo es útil, sino si hace correctamente lo que dice hacer.

La primera fue una batería de pruebas sobre el núcleo lógico, ejecutada sobre grafos construidos a mano y, por tanto, independiente de la extracción por *bytecode*. Separar ambas etapas evita confundir un defecto propio con un comportamiento de la biblioteca externa, y permite ejercitar casos límite que un caso de estudio real difícilmente ofrece: grafos vacíos, umbrales evaluados en su frontera exacta, autobucles, configuraciones inválidas y resultados internamente incoherentes. La batería reúne **234 comprobaciones** sobre el grafo, el algoritmo de Tarjan, la carga de configuración, las tres reglas, el motor, las exenciones, los cinco escenarios, los exportadores, la disposición gráfica, la comparación entre corridas, el servicio de análisis y el asistente de catálogo; todas se superan. Las del servicio y el asistente son las únicas que recorren la extracción real: verifican el reconocimiento de proyectos Maven y Gradle, el rechazo de un proyecto sin compilar, la inferencia de la raíz y la paridad entre las distintas formas de indicar un mismo sistema. Las que conciernen al dibujo verifican propiedades que una inspección visual podría pasar por alto: que el orden de las bandas se deduzca de la matriz y no del orden de declaración, que ningún par de paquetes se superponga, que las flechas que llegan a un mismo lado no compartan punto de llegada, que el SVG sea XML bien formado y que dos ejecuciones produzcan exactamente el mismo dibujo. Se construyó sin marcos de prueba externos, en coherencia con la misma restricción de ligereza que se impuso al mecanismo.

La segunda consistió en aplicar el instrumento sobre sí mismo. Si el prototipo predica una arquitectura estratificada y acíclica, resulta razonable exigirle que la practique. Con una configuración que declara sus nueve paquetes como capas y prohíbe toda dependencia ascendente, el análisis

arrojó **40 verificaciones, cero incumplimientos y una CR del** 100 %. La comprobación no sustituye a la evaluación experimental —el sistema analizado es el propio artefacto, con las limitaciones de objetividad que ello supone—, pero sí ofrece una garantía adicional de que la arquitectura interna descrita en la figura 3.2 es la realmente implementada y no una aspiración documental.

Ese mismo ejercicio produjo un hallazgo instructivo, y de él nació el mecanismo de exenciones descrito en la sección 3.7.1. Evaluado sin exenciones y con un umbral estricto, el instrumento señalaba el componente que ensambla el núcleo —primero la raíz de composición y, desde que la interfaz gráfica obligó a extraerlo, el servicio de análisis, que depende de los cinco módulos que encadena— y su módulo de reporte, que depende de los cuatro que debe representar. Ninguno de los dos es una deriva: ambos concentran dependencias por el papel que desempeñan. La salida inicial —elevar el umbral hasta que ambos dejaran de aparecer— resultaba engañosa, porque un umbral laxo aplicado a todo el sistema habría ocultado también las concentraciones reales de cualquier otro paquete. Declarar la raíz y el servicio como núcleo compartido —raíz de composición y fachada, respectivamente— y calibrar el umbral en cuatro resuelve el caso sin esa concesión, y deja la excepción a la vista en el reporte. El episodio muestra que FF3, al operar únicamente sobre la topología, es ciega al rol arquitectónico de cada paquete; volveremos sobre ello al examinar la validez de constructo.

3.12. Escenarios de evaluación

Para evaluar el mecanismo se construyeron cinco versiones de un mismo sistema sintético bajo el paquete raíz `com.caso`: un pequeño dominio de ventas organizado en las capas `presentacion`, `negocio` y `persistencia`, con un paquete de dominio compartido (`com.caso.dominio`) que no pertenece a ninguna capa.

La versión **C0** constituye la implementación de referencia y respeta la arquitectura declarada: la presentación depende del negocio, el negocio de la persistencia, y no existen dependencias en sentido contrario ni ciclos. Su grafo se muestra en la figura 3.5.

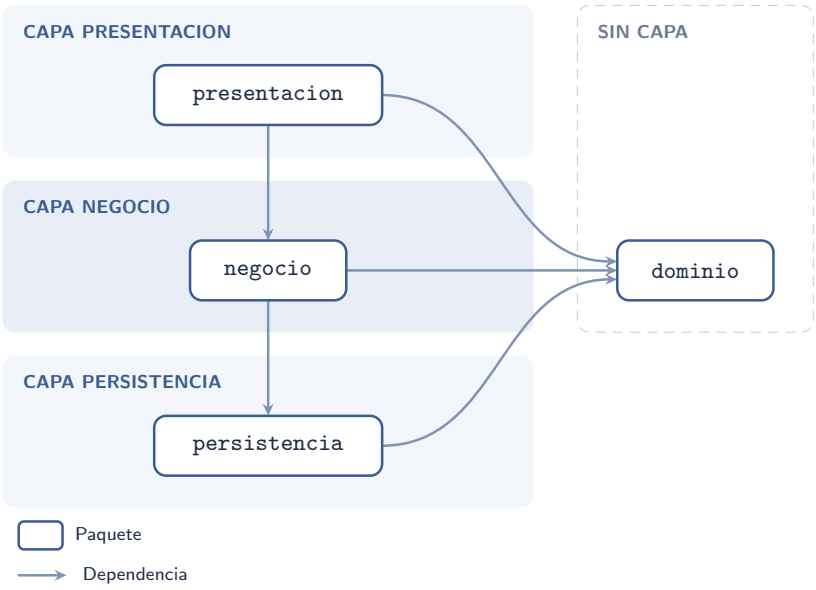


Figura 3.5: Grafo de dependencias de la implementación de referencia (C0), dibujado por el propio mecanismo: 4 paquetes y 5 dependencias, sin violaciones. Cada banda corresponde a una capa declarada; el dominio compartido, que no pertenece a ninguna, ocupa la columna lateral.

Las cuatro versiones restantes introducen derivas deliberadas. Las tres primeras las introducen **de forma aislada**, de modo que cada una active una sola *Fitness Function* y sea posible medir la sensibilidad de cada regla por separado; la cuarta las combina para observar su efecto acumulado.

- **C1 — Ruptura de capas.** El controlador de presentación accede directamente al repositorio de persistencia, saltándose la capa de negocio. La dependencia `presentacion` → `persistencia` no figura en la matriz de adyacencias y, al no invertir el sentido del flujo, no cierra ningún ciclo.
- **C2 — Ciclo de dependencia.** Un subpaquete de cálculo dentro de la capa de negocio y su paquete padre se referencian mutuamente. Como ambos pertenecen a la misma capa, la estratificación permanece intacta y solo se activa FF2. Su grafo se muestra en la figura 3.6.
- **C3 — Concentración excesiva.** Se agrega un paquete `com.caso.util` que depende de los cuatro paquetes restantes. Al no estar asignado a ninguna capa no vulnera FF1, y al no ser destino de ninguna dependencia tampoco cierra ciclos.
- **C4 — Escenario combinado.** Reúne las tres derivas: la persistencia pasa a depender del negocio y de la presentación, se forma un ciclo entre los tres paquetes de capa y reaparece el componente dios. Su grafo se muestra en la figura 3.7.

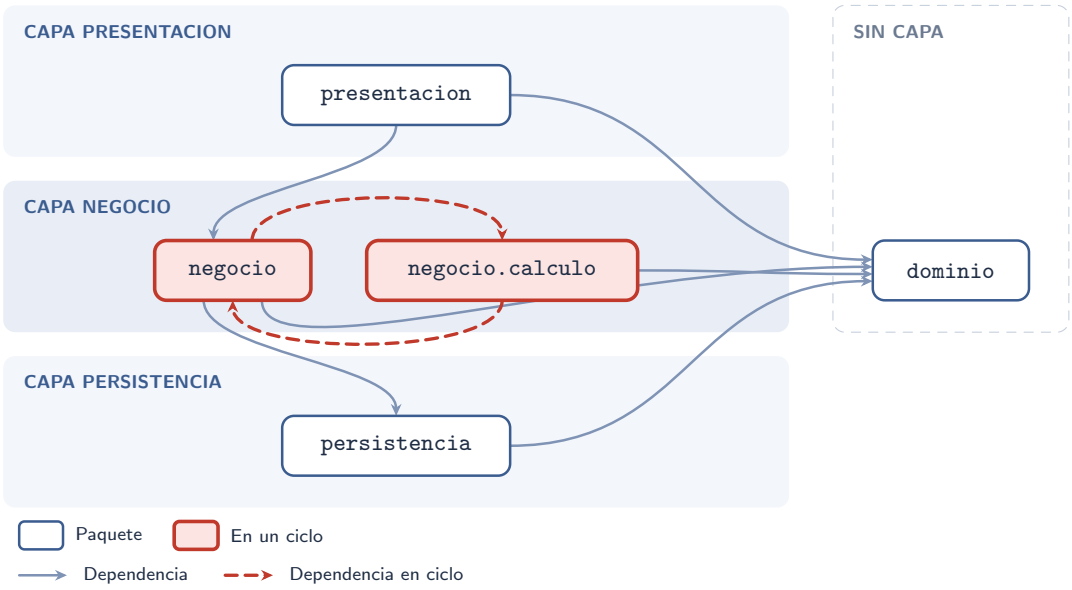


Figura 3.6: Grafo del escenario C2, dibujado por el propio mecanismo. El ciclo queda contenido en la capa de negocio: ambos paquetes comparten fila y su dependencia mutua se traza como dos arcos, uno por encima y otro por debajo. Ninguna arista remonta la estratificación.

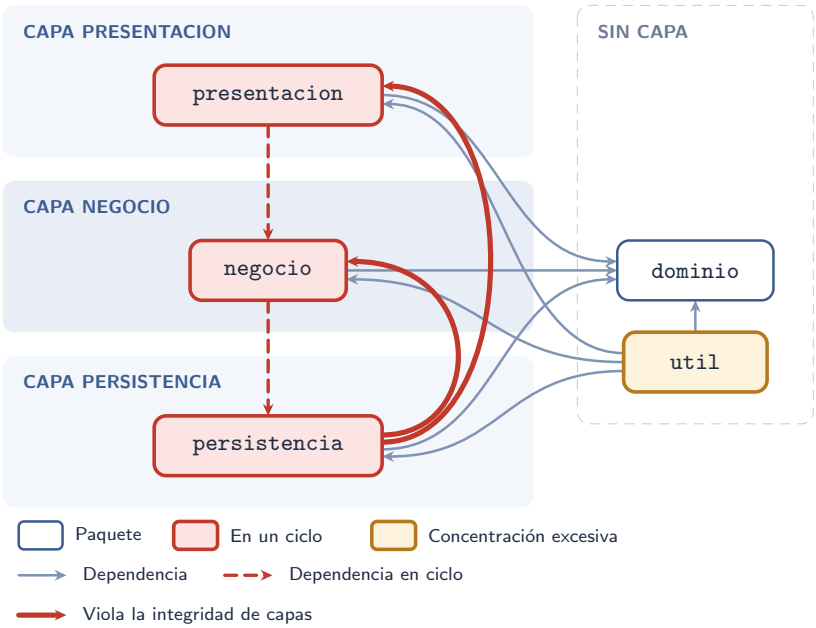


Figura 3.7: Grafo del escenario combinado (C4), dibujado por el propio mecanismo: 5 paquetes y 11 dependencias. Las dos dependencias que la persistencia dirige hacia capas superiores violan la estratificación y rodean los nodos por fuera; el ciclo que cierran se marca en los tres paquetes de capa, y el paquete que concentra dependencias salientes aparece en ámbar en la columna lateral.

Los tres escenarios aislados constituyen la principal diferencia respecto del diseño experimental inicial, que contemplaba una única versión alterada. La razón es metodológica: con las tres derivas juntas es imposible saber cuánto aporta cada regla al descenso de la CR, ni si el mecanismo sería capaz de advertir una desviación que llegara sola, que es precisamente el escenario que una PyME encontraría en la práctica.

3.13. Usos y escenarios de aplicación

La utilidad del prototipo no se agota en el experimento que lo evalúa. Dado que el artefacto se concibió como un instrumento de gobernanza para PyMES, conviene precisar qué ofrece, en qué situaciones aporta valor y en cuáles, deliberadamente, no interviene. Esta delimitación no es accesoria: define el territorio en el que el mecanismo es legítimamente aplicable y lo protege de usos para los que no fue diseñado.

En cuanto a lo que ofrece, cabe enumerar cuatro capacidades que lo distinguen de una biblioteca de reglas aislada. En primer lugar, la extracción automática del grafo de dependencias a partir del *bytecode*, que elimina la necesidad de reconstruir la estructura a mano. En segundo lugar, la verificación mediante tres *Fitness Functions* configurables, que traducen decisiones de diseño en comprobaciones ejecutables. En tercer lugar, la agregación de los resultados en una Tasa de Conformidad única y reconstruible, acompañada de una clasificación en tres niveles. Y en cuarto

lugar, un reporte que asocia cada hallazgo a la regla que lo produjo, a la evidencia estructural que lo sustenta —hasta el método y la línea que originan la dependencia— y a una severidad, y que declara explícitamente qué quedó fuera del diagnóstico. Las cuatro están disponibles por dos vías equivalentes: la línea de comandos, adecuada para automatizar el análisis y registrar su tendencia, y la interfaz gráfica, adecuada para explorar un sistema, discutir sus hallazgos en equipo y obtener un primer diagnóstico sin preparar nada.

En cuanto a los usos, el mecanismo se presta a varios escenarios que, sin ser exhaustivos, ilustran su aporte. El primero es la auditoría puntual de conformidad: ante la sospecha de que una arquitectura ha dejado de respetar sus capas, una ejecución única ofrece un panorama inmediato del estado estructural sin exigir una revisión manual exhaustiva. El segundo es la señal temprana por iteración: si el análisis se ejecuta al cierre de cada ciclo de desarrollo, el equipo puede advertir una desviación apenas se introduce, cuando todavía es barata de revertir, en lugar de descubrirla meses después (Klein et al., 2022). El tercero es la priorización de la refactorización: al clasificar las violaciones por severidad y por dimensión, el reporte orienta al líder técnico sobre qué intervenciones atacar primero, un apoyo valioso cuando los recursos de mantenimiento son escasos (Hamed et al., 2023). El cuarto es el complemento de la documentación de decisiones: la Tasa de Conformidad y las violaciones detectadas constituyen el insumo objetivo contra el cual contrastar los *Architecture Decision Records*, cerrando el ciclo entre lo decidido y lo efectivamente construido (AOE Technology Radar, 2026).

En cuanto al contexto de aplicación, el instrumento está pensado para el perfil que motivó su diseño: una PyME de software que desarrolla en Java, cuya arquitectura se organiza en capas y que carece de documentación arquitectónica actualizada. Resulta pertinente en tres situaciones típicas. La primera es la incorporación de nuevos miembros al equipo, escenario en el que el reporte sirve como fotografía verificable de la estructura real del sistema. La segunda es la preparación de una entrega relevante, en la que el mecanismo actúa como una comprobación de salud estructural antes de comprometer una versión. La tercera es el acompañamiento de una refactorización en curso, donde la comparación entre corridas sucesivas permite comprobar si el esfuerzo invertido se traduce efectivamente en mejora estructural.

Conviene, sin embargo, deslindar con honestidad un cuarto escenario que la intuición sugiere pero que el prototipo, en su estado actual, no sostiene: el diagnóstico inicial de un sistema legado que ya arrastra deriva acumulada (Anthony et al., 2024). El obstáculo no es técnico sino de utilidad práctica. Un sistema que lleva años desviándose produce, en su primera ejecución, un volumen de hallazgos que ningún equipo puede accionar, y la experiencia con herramientas de análisis estático muestra que el desenlace habitual de esa avalancha es que el instrumento se desactive. Atender ese escenario exige poder fijar el estado actual como línea de partida y reportar únicamente las violaciones *nuevas*, un mecanismo que el alcance aprobado no contemplaba y que se plantea, con la evidencia recogida, en el capítulo de conclusiones. El instrumento resulta hoy más adecuado para preservar una arquitectura que todavía se respeta que para diagnosticar una que ya se perdió.

Con la misma claridad conviene delimitar lo que el prototipo no hace, en estricta correspondencia con el alcance aprobado en el anteproyecto. No corrige automáticamente las derivas que detecta: su propósito es diagnosticar, clasificar y aportar evidencia para la decisión humana, no reescribir

el código. No está integrado como producto de producción ni como etapa de una tubería CI/CD organizacional; la opción de terminar con un código de salida distinto de cero es una afordancia para una adopción futura, no una integración entregada. No soporta múltiples ecosistemas tecnológicos en una misma ejecución, ni realiza análisis dinámico en tiempo de ejecución. Estas fronteras no restan valor al mecanismo; lo acotan al propósito para el que fue evaluado y evitan sobrevender capacidades que no posee. La contribución del prototipo no reside en competir con plataformas de verificación industrial, sino en ofrecer un punto de partida liviano, comprensible y reproducible para que un equipo pequeño comience a gobernar su arquitectura con evidencia (Modawski and Wolniak, 2025).

3.14. Trazabilidad entre requisitos e implementación

La tabla 3.2 vincula cada requisito con el componente que lo materializa y con la evidencia que permite comprobar su satisfacción, de modo que la verificación del cumplimiento no dependa de la lectura del código.

Tabla 3.2: Trazabilidad entre requisitos, implementación y evidencia de cumplimiento.

Req.	Componente que lo materializa	Evidencia de cumplimiento
RF1	`ConstructorGrafo`, `GrafoDependencias`	Conteo de paquetes y dependencias declarado en cada reporte; grafos exportados en formato DOT.
RF2	`ReglaIntegridadCapas`	Detección de la deriva en C1 y en C4; ausencia de hallazgos en C0, C2 y C3.
RF3	`ReglaCiclos`, `Tarjan`, `BuscadorCiclos`	Detección del ciclo en C2 y en C4, con la ruta cíclica concreta en la evidencia.
RF4	`ReglaConcentracion`	Detección del componente dios en C3 y en C4, con los valores de Ce y Ca.
RF5	`MotorConformidad`, `GeneradorReporte`	Tabla comparativa de los cinco escenarios; reportes legibles con CR y estado.
RF6	`Violacion`, evidencia por arista, sección de alcance del reporte	Hallazgos con método y línea de origen; declaración de reglas, umbrales y exclusiones en todo reporte.
RF7	`DisposicionGrafo`, `ExportadorSvg`, `ExportadorTikz`	Figuras 3.5, 3.6 y 3.7, generadas por el mecanismo; SVG por escenario consultable en cualquier navegador.
RF8	`VentanaPrincipal`, `ResolutorProyecto`, `Analisis`	Figura 3.3; pruebas de paridad entre la carpeta del proyecto, el directorio de clases y el `.jar`; autoanálisis que impide a la interfaz eludir el servicio.
RF9	`VentanaAsistente`, `Conversacion`, `Propuesta`, `Asistencia`	Figura 3.4; pruebas contra un servidor que imita las API de los modelos; la propuesta se valida con `CargadorConfiguracion` y su previsualización sobre C4 reproduce la CR del experimento.
RNF1	`CargadorConfiguracion`, `Exenciones`	Los cinco escenarios, el autoanálisis y la aplicación a un sistema externo se ejecutan con el mismo binario y configuraciones distintas; el reporte enumera las exenciones aplicadas.
RNF2	Ausencia de dependencias más allá de *ArchUnit*	Compilación y ejecución con un JDK, sin Maven ni red; batería de pruebas sin marco externo.
RNF3	`GeneradorReporte`	Reporte con conformidad local por regla, evidencia por hallazgo y declaración de alcance.
RNF4	Exportadores JSON, CSV, SVG, TikZ y DOT	Ejecuciones repetidas con resultados idénticos; evidencia versionable y verificable por terceros.

3.15. Resumen del capítulo

Este capítulo documentó el diseño y la implementación del mecanismo. Se describió la propuesta como un instrumento de tres etapas —extracción del grafo, evaluación mediante *Fitness Functions* y clasificación agregada—, se precisaron sus requisitos, se presentó una arquitectura de cinco módulos con dependencias acíclicas, a la que un servicio de análisis, una interfaz gráfica y un asistente para

redactar el catálogo dan acceso, y se formalizó el catálogo de reglas que constituye su núcleo normativo. Se justificó, además, la operacionalización de la Tasa de Conformidad y se dejaron explícitas las tres limitaciones que su formulación proporcional arrastra. La implementación en Java se verificó mediante una batería de 234 comprobaciones y mediante la aplicación del propio instrumento sobre sí mismo. Sobre esta base, y con los cinco escenarios ya construidos, el capítulo siguiente describe cómo se evaluó el mecanismo y qué resultados se obtuvieron.

Capítulo 4

Evaluación

Este capítulo presenta la evaluación del mecanismo. Se describe primero el diseño del experimento y la matriz de validación que fija de antemano lo que debía observarse; se exponen después los resultados obtenidos sobre los cinco escenarios; se analiza a continuación un hallazgo que obligó a refinar el criterio de clasificación, y se cierra contrastando los resultados con los indicadores comprometidos en la metodología y examinando las amenazas a la validez de la evidencia producida.

4.1. Diseño de la evaluación

La evaluación se planteó como un experimento controlado de caso único, la modalidad que la investigación en diseño reserva para demostrar la viabilidad de un artefacto antes de someterlo a validación en campo (Hevner et al., 2004; Peffers et al., 2007). Su propósito fue determinar si el mecanismo construido detecta las derivas arquitectónicas que se le presentan y si clasifica de forma coherente el estado de conformidad del sistema evaluado.

La hipótesis de trabajo fue que, al aplicar el mecanismo sobre una implementación de referencia y sobre versiones deliberadamente alteradas, la primera obtendría una conformidad plena sin hallazgos, mientras que cada versión alterada activaría la *Fitness Function* correspondiente a la deriva inducida y vería descender su Tasa de Conformidad.

Como variable independiente se manipuló la presencia y el tipo de deriva en el código, según los cinco escenarios descritos en el capítulo anterior. Las variables dependientes observadas fueron cuatro: los hallazgos reportados por cada *Fitness Function*, la Tasa de Conformidad resultante, el estado arquitectónico asignado y la evidencia estructural asociada a cada hallazgo. Todas las demás condiciones se mantuvieron constantes —el catálogo de reglas de la sección 3.4, los umbrales, la versión del prototipo y el entorno de ejecución—, de modo que las diferencias observadas puedan atribuirse a la variable manipulada y no a variaciones del procedimiento (Wohlin et al., 2012).

La incorporación de los tres escenarios de deriva aislada (C1, C2 y C3) constituye la decisión de diseño más relevante de esta evaluación. Un único escenario que acumulase las tres derivas permitiría comprobar que el mecanismo reacciona, pero no cuánto aporta cada regla a esa reacción ni si el instrumento advertiría una desviación que llegara sola, que es el caso que una PyME encontraría en la práctica. Aislar las derivas convierte el experimento en una prueba de sensibilidad por dimensión y no en una simple comprobación de funcionamiento.

4.1.1. Matriz de validación

Antes de ejecutar el experimento se fijó la matriz de validación de la tabla 4.1, que declara para cada escenario qué reglas debían activarse y cuáles debían permanecer en silencio. Establecerla de antemano cumple una función metodológica precisa: impide que el resultado observado se reinterprete después como el esperado, y convierte los falsos negativos y los falsos positivos en desviaciones verificables y no en apreciaciones.

Tabla 4.1: Matriz de validación: comportamiento esperado de cada *Fitness Function* por escenario.

Escenario	FF1	FF2	FF3	Justificación del comportamiento esperado
C0 · base	—	—	—	La implementación respeta la matriz de adyacencias, es acíclica y ningún paquete supera el umbral.
C1 · capas	✓	—	—	El salto `presentacion` → `persistencia` no está autorizado, pero no invierte el flujo ni cierra ciclos.
C2 · ciclo	—	✓	—	El ciclo queda contenido dentro de la capa de negocio, de modo que la estratificación no se vulnera.
C3 · concentración	—	—	✓	El paquete `util` no pertenece a ninguna capa y no es destino de ninguna dependencia.
C4 · combinado	✓	✓	✓	Las tres derivas actúan simultáneamente sobre el mismo sistema.

4.1.2. Criterios de éxito y procedimiento

Para juzgar el objetivo específico OBJ4 se estableció un criterio cuantitativo fijado en el anteproyecto: el mecanismo debía detectar al menos el 80 % de las derivas inducidas. Con seis derivas repartidas entre los cuatro escenarios alterados, el criterio exigía detectar al menos cinco. Se añadió una segunda condición, de signo contrario y de igual importancia: la implementación de referencia no debía producir ningún hallazgo, pues un instrumento que alarma sin motivo resulta tan inútil como uno que calla ante el problema.

El procedimiento se ejecutó íntegramente en un entorno de desarrollo local, sin dependencias de servicios externos, en coherencia con RNF2. Cada ejecución recibió los mismos dos insumos —el directorio de clases compiladas y el archivo de configuración— y se automatizó en un guion único que recorre los cinco escenarios y deposita la evidencia en archivos verificables: el reporte legible, la evidencia estructurada en JSON, el grafo en formato DOT y los hallazgos tabulados en CSV. Las ejecuciones se repitieron para comprobar que el resultado fuese idéntico, verificando así el requisito de reproducibilidad. El anexo A detalla los pasos necesarios para reproducir el experimento completo.

4.2. Resultados de la evaluación

La tabla 4.2 resume el resultado agregado sobre los cinco escenarios, con los valores de la Tasa de Conformidad calculados según la ecuación 3.1 y el estado arquitectónico deducido de sus umbrales.

Tabla 4.2: Resultados agregados del mecanismo sobre los cinco escenarios.

Escenario	Paq.	Dep.	Verif.	Incum.	CR	Estado según CR
C0 · base	4	5	10	0	100,00 %	Aceptable
C1 · capas	4	6	11	1	90,91 %	Aceptable
C2 · ciclo	5	8	12	2	83,33 %	En observación
C3 · concentración	5	9	12	1	91,67 %	Aceptable
C4 · combinado	5	11	14	6	57,14 %	No conforme

4.2.1. Contraste con la matriz de validación

La tabla 4.3 contrasta el comportamiento esperado con el efectivamente observado. La coincidencia es completa: en los cinco escenarios se activaron exactamente las reglas previstas, sin omisiones ni activaciones espurias.

Tabla 4.3: Contraste entre el comportamiento esperado y el observado. Cada celda indica el número de hallazgos reportados por la regla.

	Esperado			Observado				
Escenario	FF1	FF2	FF3	FF1	FF2	FF3	**Total**	**Veredicto**
C0 · base	—	—	—	0	0	0	0	Coincide
C1 · capas	✓	—	—	1	0	0	1	Coincide
C2 · ciclo	—	✓	—	0	1	0	1	Coincide
C3 · concentración	—	—	✓	0	0	1	1	Coincide
C4 · combinado	✓	✓	✓	2	1	1	4	Coincide

En C4, la *Fitness Function* de integridad de capas reportó dos hallazgos —las dependencias indebidas de `persistencia` hacia `negocio` y hacia `presentacion`— porque el escenario introduce dos aristas prohibidas y cada una constituye una violación independiente. La tabla 4.4 detalla los hallazgos de ese escenario.

Tabla 4.4: Hallazgos detectados en el escenario combinado (C4).

Fitness Function	**Hallazgo**	**Severidad**
Integridad de capas	Dependencia no permitida `persistencia` → `negocio`	Mayor
Integridad de capas	Dependencia no permitida `persistencia` → `presentacion`	Mayor
Ausencia de ciclos	Ciclo entre `presentacion`, `negocio` y `persistencia`	Crítica
Concentración	`com.caso.util` concentra cuatro dependencias salientes (umbral: 3)	Mayor

Conviene aclarar una asimetría que la tabla podría ocultar: C4 reporta cuatro hallazgos pero seis incumplimientos. La diferencia procede de FF2, que cuenta un incumplimiento por cada paquete atrapado en el ciclo —tres, en este caso— aunque los agrupe en un único hallazgo legible. La

decisión es deliberada: la penalización debe crecer con la extensión del enredo, mientras que el reporte debe presentar el ciclo como el problema único que en realidad es.

4.2.2. Sensibilidad por dimensión

Los escenarios aislados permiten observar el comportamiento de cada regla por separado. La tabla 4.5 muestra la conformidad local de cada *Fitness Function*, es decir, la proporción de sus propias verificaciones que el sistema superó.

Tabla 4.5: Conformidad local por *Fitness Function* en cada escenario (verificaciones superadas sobre verificaciones realizadas).

Escenario	**FF1 — Capas**	**FF2 — Ciclos**	**FF3 — Concentración**
C0 · base	2/2 (100 %)	4/4 (100 %)	4/4 (100 %)
C1 · capas	2/3 (66,7 %)	4/4 (100 %)	4/4 (100 %)
C2 · ciclo	2/2 (100 %)	3/5 (60,0 %)	5/5 (100 %)
C3 · concentración	2/2 (100 %)	5/5 (100 %)	4/5 (80,0 %)
C4 · combinado	2/4 (50,0 %)	2/5 (40,0 %)	4/5 (80,0 %)

La lectura por filas confirma el aislamiento buscado: en cada escenario de deriva única solo desciende la conformidad de la regla correspondiente, mientras las otras dos permanecen en el 100 %. La lectura por columnas revela algo más sutil: la conformidad local reacciona con claridad —FF2 cae al 60 % en C2, FF1 al 66,7 % en C1—, mucho más de lo que reacciona la CR global de esos mismos escenarios. Esa discrepancia entre la señal local y la global es la que motiva el análisis de la sección siguiente.

4.2.3. Trazabilidad del cálculo

El valor del 57,14 % obtenido en C4 no es una puntuación opaca: corresponde al cociente entre las ocho verificaciones que el sistema superó y las catorce que las tres reglas ejecutaron. Cada incumplimiento resta directamente a ese cociente, de modo que la cifra final puede reconstruirse a mano a partir de la tabla 4.5. Esta propiedad de trazabilidad es, en sí misma, un resultado relevante: un equipo de desarrollo puede explicar con precisión por qué su proyecto obtuvo una determinada tasa, lo que convierte el diagnóstico en un insumo negociable y no en un veredicto inapelable.

La evidencia acompaña a cada hallazgo hasta su origen. En C1, por ejemplo, el reporte no se limita a señalar la arista infractora: identifica el campo, el constructor y el método del controlador que la producen, con el archivo y la línea correspondientes. Esa precisión, incorporada durante la evaluación como requisito RF6, transforma el diagnóstico en una lista de intervenciones concretas.

4.3. Un hallazgo sobre la métrica: la dilución de las violaciones localizadas

Los escenarios aislados produjeron un resultado que el diseño experimental original no habría revelado. En C1 y en C3 el mecanismo detectó correctamente la deriva inducida —FF1 y FF3 reportaron su hallazgo con la evidencia correspondiente—, pero la Tasa de Conformidad se mantuvo en 90,91 % y 91,67 % respectivamente, por encima del umbral del 85 %. En consecuencia, el estado arquitectónico asignado fue *aceptable* en ambos casos, pese a existir una violación real y documentada.

No se trata de un defecto de implementación sino de una consecuencia matemática de la formulación proporcional anticipada en la sección 3.5: una violación aislada representa una fracción pequeña del total de verificaciones, y esa fracción se reduce conforme el sistema crece. En los escenarios evaluados, con once o doce verificaciones, un incumplimiento apenas mueve la métrica nueve puntos. En un sistema de doscientos paquetes, el mismo incumplimiento la movería menos de un punto. La CR mide con fidelidad la **extensión** del deterioro, pero es estructuralmente ciega a su **gravedad**.

La consecuencia práctica es seria para el propósito del proyecto. Un instrumento pensado como señal temprana debería alertar precisamente cuando la primera desviación aparece, que es cuando corregirla resulta barata; sin embargo, el criterio basado solo en la CR guarda silencio hasta que las desviaciones se acumulan. El riesgo se agrava porque el veredicto *aceptable* podría interpretarse como ausencia de problemas, y no como lo que realmente significa.

Merece subrayarse que este riesgo no era desconocido: el anteproyecto lo había anticipado al identificar la sobreinterpretación de la CR como uno de los riesgos éticos del proyecto y al advertir que la métrica «no debe reemplazar el juicio técnico». Lo que la evaluación aporta no es la intuición del riesgo, sino la medida de su magnitud y una respuesta verificable.

4.4. Refinamiento del artefacto

La investigación en diseño prescribe que los resultados de la evaluación retroalimenten el artefacto en un nuevo ciclo, en lugar de limitarse a describir sus carencias (Hevner et al., 2004; Peffers et al., 2007). Siguiendo ese principio se introdujo un segundo criterio de clasificación, complementario y no sustitutivo del primero.

El razonamiento es el siguiente. La Tasa de Conformidad responde a la pregunta «¿qué proporción del sistema incumple?»; la severidad del peor hallazgo responde a «¿qué tan grave es lo que incumple?». Ambas preguntas son legítimas y ninguna subsume a la otra: un sistema puede tener muchas violaciones leves o una sola crítica, y esos dos estados requieren respuestas distintas. El refinamiento consiste en evaluar las dos y adoptar como veredicto el más restrictivo, según la correspondencia siguiente:

- una violación **crítica** —un ciclo de dependencia— exige el estado *no conforme*;

- una violación **mayor** —una ruptura de capas o una concentración excesiva— exige, al menos, el estado *en observación*;
- la ausencia de hallazgos permite el estado *aceptable*.

Dos precisiones delimitan el alcance de este cambio. La primera es que **la Tasa de Conformidad no se modifica**: se sigue calculando igual, se sigue interpretando con los umbrales del 85 % y el 70 %, y se sigue reportando siempre, de modo que la formulación aprobada en el anteproyecto permanece intacta y verificable. La segunda es que el criterio es **configurable**: el modo `cr` reproduce el comportamiento original y el modo `cr+severidad` aplica el refinamiento, lo que permite comparar ambos sobre la misma evidencia en lugar de sustituir uno por otro sin posibilidad de contraste. El reporte, además, advierte de forma explícita cuando la severidad ha endurecido el veredicto que la CR habría emitido por sí sola.

La tabla 4.6 muestra el resultado de reejecutar el experimento completo bajo ambos criterios.

Tabla 4.6: Clasificación de los cinco escenarios bajo ambos criterios. La columna «esperado» recoge el estado que el escenario merece según las derivas efectivamente inducidas.

Escenario	**CR**	**Modo `cr`**	**Modo `cr+severidad`**	**Esperado**
C0 · base	100,00 %	Aceptable	Aceptable	Aceptable
C1 · capas	90,91 %	Aceptable	En observación	En observación o peor
C2 · ciclo	83,33 %	En observación	No conforme	No conforme
C3 · concentración	91,67 %	Aceptable	En observación	En observación o peor
C4 · combinado	57,14 %	No conforme	No conforme	No conforme

El criterio original clasifica correctamente tres de los cinco escenarios; el refinado, los cinco. La mejora se concentra, como era de esperar, en los escenarios de deriva aislada, que son justamente aquellos en los que un mecanismo de detección temprana debe demostrar su valor. Obsérvese, además, que el refinamiento solo puede endurecer el veredicto y nunca relajarlo, propiedad que la batería de pruebas verifica de forma explícita: incorporar la severidad no puede hacer que un sistema problemático parezca sano.

Queda pendiente una discusión que el refinamiento abre y no cierra. Que un único ciclo baste para declarar un sistema *no conforme* es una posición defendible —los ciclos son la estructura más dañina de las tres— pero también exigente para un sistema legado que arrastre varios. Por esa razón el criterio se dejó configurable y no impuesto: cada organización debe decidir cuánta severidad tolera, del mismo modo que decide sus umbrales. Lo que el mecanismo garantiza es que esa decisión quede declarada y sea auditable, no que sea la misma para todos.

4.5. Contraste con los indicadores de la metodología

La metodología aprobada en el anteproyecto definió cuatro indicadores para juzgar el desempeño del mecanismo. Contrastar los resultados con cada uno de ellos es lo que convierte un experimento en

una evaluación con criterios explícitos, en lugar de una demostración de que el prototipo «funciona». La tabla 4.7 resume esa correspondencia.

Tabla 4.7: Contraste entre los indicadores metodológicos y los resultados observados.

Indicador	**Definición**	**Resultado observado**
Cobertura de detección	Proporción de derivas inducidas que el mecanismo identifica.	Detectó las seis derivas repartidas entre los cuatro escenarios alterados: cobertura del 100 %, superior al criterio de éxito del 80 % fijado para OBJ4.
Falsos positivos en la línea base	Verificación de que el mecanismo no marque como anómalo un estado estructural esperado.	C0 obtuvo conformidad plena y cero hallazgos. Ninguna regla se activó fuera de los escenarios previstos por la matriz de validación, de modo que tampoco hubo falsos positivos en los escenarios alterados.
Variación de la Tasa de Conformidad	Comprobación de que la métrica sea sensible a la degradación introducida.	La CR descendió en los cuatro escenarios alterados, de forma monótona respecto del número de incumplimientos (100 % → 91,67 % → 90,91 % → 83,33 % → 57,14 %). Es sensible, pero su magnitud resultó insuficiente para clasificar correctamente las derivas aisladas, lo que motivó el refinamiento de la sección anterior.
Capacidad explicativa del reporte	Presencia de la regla violada, su ubicación y evidencia estructural suficiente para interpretación humana.	Cada hallazgo asocia la regla, los paquetes implicados, la severidad y la evidencia hasta el método y la línea que originan la dependencia; los ciclos incluyen su ruta concreta. El reporte declara además reglas, umbrales, paquetes sin capa y exclusiones del diagnóstico.

Este contraste cumple una doble función. Por un lado, demuestra que el experimento se juzgó contra los mismos criterios que el anteproyecto había comprometido, sin introducir criterios a posteriori. Por otro, muestra que el trabajo no se limitó a confirmar lo que ya se esperaba: el tercer indicador reveló una limitación real de la métrica, y esa limitación se atendió dentro del alcance aprobado en lugar de posponerse como trabajo futuro.

4.6. Aplicación a un sistema externo

Todo lo anterior se obtuvo sobre escenarios construidos para el experimento. Cabe entonces una pregunta que el diseño controlado no puede responder por sí mismo: ¿qué ocurre cuando el mecanismo se enfrenta a código que nadie preparó para él? Para responderla se aplicó el instrumento sobre *ArchUnit* en su versión 1.3.0, una biblioteca de uso extendido cuyo código no fue escrito por el autor ni con este trabajo en mente. El análisis se realizó directamente sobre el archivo `.jar` con

que la biblioteca se distribuye, sin extraerlo, y las configuraciones empleadas se conservan en el repositorio para que el ejercicio pueda reproducirse.

Conviene delimitar el estatus de este ejercicio antes de exponer sus resultados. **No constituye un segundo caso de estudio ni forma parte del experimento controlado**, cuyo alcance —un único sistema— permanece tal como el anteproyecto lo aprobó. Es una demostración de aplicabilidad, del mismo tipo que el autoanálisis del capítulo anterior: muestra que el mecanismo opera sobre código real y a qué se enfrenta al hacerlo, sin pretender que de ello se siga una validación externa.

Una segunda precisión es de justicia hacia el sistema analizado. La estratificación evaluada —`base` ← `core` ← `lang` ← `library`— es una **hipótesis** formulada a partir de los nombres de sus módulos, no una arquitectura que el proyecto declare. Cualquier desviación que se reporte lo es respecto de esa suposición, y no necesariamente un defecto de *ArchUnit*.

El sistema aporta 923 clases propias repartidas en 22 paquetes. El análisis completo tarda **1,2 segundos**, magnitud que lo hace compatible con una ejecución en cada integración. La tabla 4.8 contrasta el diagnóstico obtenido sin exenciones con el obtenido tras declararlas.

Tabla 4.8: Diagnóstico de *ArchUnit* 1.3.0 antes y después de declarar las exenciones.

	Sin exenciones	**Con exenciones**
Paquetes en el grafo	43	22
Dependencias extraídas	222	107
FF1 — Integridad de capas	61/61	61/61
FF2 — Ciclos (incumplimientos)	12	9
tamaño del mayor ciclo	7 paquetes	4 paquetes
FF3 — Concentración (hallazgos)	14	5
Tasa de Conformidad	82,31 %	86,41 %
Estado según CR	En observación	Aceptable
Estado final (con severidad)	No conforme	No conforme

De estos números se desprenden tres lecturas, y las tres importan por razones distintas.

La regla de capas resultó exacta. FF1 realizó 61 verificaciones y las superó todas, en ambas configuraciones y sin un solo falso positivo. Dos conclusiones se siguen de ello: que la hipótesis sobre la arquitectura de *ArchUnit* era correcta —su estratificación se respeta de forma estricta— y que la regla es capaz de validar una arquitectura real sin ruido. De las tres dimensiones del catálogo, esta es la que se muestra lista para uso industrial.

Sin exenciones, casi la mitad del grafo era código ajeno. *ArchUnit* empaqueta Guava y ASM bajo su propio prefijo, de modo que la restricción a la raíz —pensada para dejar fuera a las bibliotecas de terceros— no los excluye. Peor aún, esos paquetes entran en el grafo incluso cuando sus clases no se importan, porque basta con que alguien las referencie para que aparezca la arista.

El efecto es cuantificable: declarar las exenciones elimina **9 de los 14** hallazgos de concentración, es decir, cerca de dos tercios eran ruido, y reduce el ciclo mayor de siete paquetes a cuatro, porque el paquete de anotaciones al que todo el sistema referencia actuaba como puente entre módulos que entre sí no se alcanzan. La Tasa de Conformidad asciende en consecuencia 4,10 puntos porcentuales y cruza el umbral de aceptación.

Los hallazgos reales sobreviven. Los tres ciclos permanecen tras las exenciones, y el veredicto final sigue siendo *no conforme* por su severidad. Esta es, probablemente, la observación más importante del ejercicio: el mecanismo de exenciones retira ruido sin silenciar hallazgos, que es la única forma en que una excepción puede ser legítima. Si las exenciones hubieran hecho desaparecer también los ciclos, habría motivos para sospechar que el instrumento se había convertido en un complaciente.

La figura 4.1 muestra el grafo resultante tal como lo dibujó el mecanismo. Ofrece una lectura que los números no transmiten: las cuatro capas aparecen en el orden correcto aunque la configuración las declaró de abajo arriba, los tres ciclos se concentran en las capas `core` y `lang` —no en la de reglas predefinidas—, y el paquete `base`, del que dependen casi todos los demás, se reconoce de inmediato por su anchura. El dibujo es denso porque el sistema lo es: ciento siete dependencias entre veintidós paquetes no admiten una representación despejada, y la figura no lo disimula.

Queda por señalar qué *no* establece este ejercicio. Se trata de un solo sistema, analizado por el propio autor, bajo una arquitectura supuesta y sin contraste con el criterio de quienes lo desarrollaron. Sirve para mostrar que el mecanismo opera sobre código real, a qué velocidad y con qué clase de dificultades; no sustituye la validación externa sobre repositorios industriales que el capítulo de conclusiones plantea como línea prioritaria de trabajo.

4.7. Amenazas a la validez

Toda evaluación empírica debe examinar con honestidad los límites de la evidencia que produce. El análisis sigue la clasificación habitual en investigación empírica en ingeniería de software (Wohlin et al., 2012; Runeson and Höst, 2009).

Validez interna. La principal amenaza proviene de la naturaleza sintética de los escenarios: al haber sido las derivas introducidas por el propio investigador, existe el riesgo de que el mecanismo se haya ajustado, aun de forma no intencional, a las particularidades de esos casos. Se aplicaron tres contenciones. Primera, las reglas se mantuvieron genéricas y se declararon por configuración, de modo que no codifican conocimiento del caso alterado; el mismo binario evaluó los cinco escenarios y el propio prototipo, con configuraciones distintas. Segunda, la matriz de validación se fijó antes de ejecutar el experimento, lo que impide reinterpretar a posteriori lo observado. Tercera, el comportamiento esperado se codificó además en la batería de pruebas, que verifica los valores de la CR sobre grafos construidos a mano, con independencia de la extracción por *bytecode*: si la herramienta de análisis estático produjera un grafo distinto del previsto, la discrepancia se manifestaría

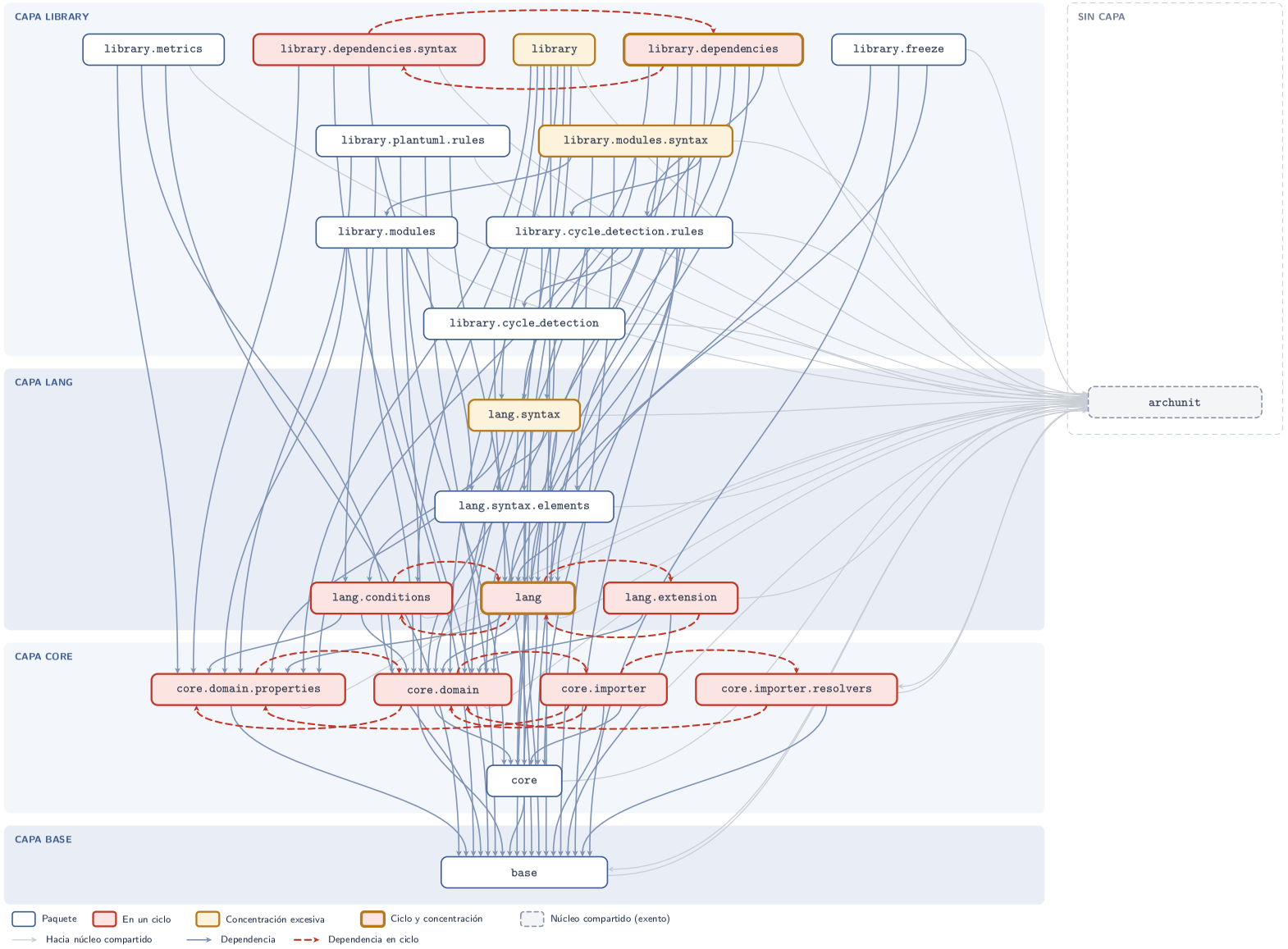


Figura 4.1: Grafo de dependencias de *ArchUnit* 1.3.0 con las exenciones declaradas, dibujado por el propio mecanismo a partir del archivo `.jar`. Las bandas siguen la estratificación hipotética `library` → `lang` → `core` → `base`; el núcleo de anotaciones exento ocupa la columna lateral y sus dependencias se dibujan atenuadas.

en lugar de pasar inadvertida. Una amenaza que no se logra contener por completo es que el refinamiento de la clasificación se diseñó *después* de observar el comportamiento de C1 y C3; aunque su formulación es independiente de esos escenarios —no menciona capas, ciclos ni concentración, sino solo severidades—, su validación sobre los mismos casos que lo motivaron no constituye evidencia independiente.

Validez externa. La generalización está limitada por el tamaño y la artificialidad de los escenarios. Cinco paquetes y once dependencias distan mucho de un sistema industrial, y los resultados demuestran la viabilidad del enfoque sin autorizar a extrapolar su desempeño a proyectos con arquitecturas heterogéneas, múltiples lenguajes o volúmenes de código muy superiores. El autoanálisis del prototipo —seis paquetes, catorce dependencias, veinticuatro verificaciones— aporta un segundo sistema real, aunque igualmente pequeño y desarrollado por el mismo autor. La aplicación a *ArchUnit* descrita en la sección 4.6 añade un tercero que sí es ajeno y de un orden de magnitud mayor, y con ello despeja una duda concreta: el mecanismo opera sobre código industrial sin degradarse, en poco más de un segundo. Pero no resuelve la amenaza de fondo, por tres razones que conviene no disimular: sigue siendo un único sistema, la arquitectura contra la que se le evaluó fue supuesta por el propio autor en lugar de declarada por sus desarrolladores, y nadie con conocimiento del sistema confirmó si los hallazgos corresponden a deriva real o a decisiones deliberadas. Hay, además, un efecto que el tamaño reducido de los escenarios oculta: la dilución descrita en la sección 4.3 se agrava con la escala, de modo que en sistemas grandes cabe esperar que el criterio basado solo en la CR sea aún menos discriminante que lo observado aquí. La generalización exige una validación sobre repositorios industriales con la participación de quienes los mantienen, que se plantea como trabajo futuro.

Validez de constructo. La pregunta es si las variables observadas representan adecuadamente el fenómeno que se pretende medir. Operacionalizar la deriva arquitectónica en solo tres dimensiones es, necesariamente, una aproximación parcial de un fenómeno complejo: un sistema puede obtener una CR del 100 % y padecer derivas que el catálogo no cubre, razón por la cual el reporte declara explícitamente qué quedó fuera del diagnóstico. A ello se suma una limitación específica de FF3 que el autoanálisis puso de manifiesto: al operar solo sobre la topología, la regla no distingue un componente dios de un concentrador legítimo, como una raíz de composición o una fachada, y por tanto su umbral no es un valor universal sino un parámetro que cada organización debe calibrar.

Las exenciones descritas en la sección 3.7.1 atenúan esa ceguera, pero no la eliminan, y al introducirlas aparece una amenaza nueva que debe declararse con franqueza. El mecanismo sigue sin ser capaz de reconocer por sí mismo el papel de un paquete: se limita a aplicar el que alguien le declaró. Esto traslada el juicio a quien configura, y con ello abre la posibilidad de que una exención mal concedida oculte una concentración o un ciclo perfectamente reales. La evidencia recogida sobre *ArchUnit* sugiere que el riesgo no se materializó allí —los tres ciclos sobrevivieron a las exenciones—, pero un solo caso favorable no es garantía. Las salvaguardas disponibles son de procedimiento y no técnicas: que la exención se declare en la configuración y no en el código, que el reporte la enumere siempre, y que su concesión quede sujeta a la misma revisión que cualquier

otra decisión arquitectónica.

Por último, la asignación de paquetes a capas se resuelve por prefijo y en el orden en que las capas se declaran, lo que exige cuidado cuando una capa es prefijo de otra.

Validez de conclusión. Las conclusiones se apoyan en la transparencia del cálculo: al ser la CR un cociente explícito y reproducible, cualquier afirmación puede auditarse reconstruyendo el diagnóstico paso a paso, y la evidencia se conserva en formatos verificables por terceros. No obstante, el carácter determinista del experimento —cinco escenarios, una configuración, un sistema— impide todo tratamiento estadístico. Los resultados deben leerse como evidencia de viabilidad y de sensibilidad por dimensión, no como una estimación de desempeño generalizable.

4.8. Resumen del capítulo

Este capítulo presentó el diseño de la evaluación, concebido como un experimento controlado con una matriz de validación fijada de antemano y con criterios de éxito explícitos. El mecanismo detectó las seis derivas inducidas en los cuatro escenarios alterados, sin producir un solo hallazgo sobre la implementación de referencia, superando así el criterio del 80 % comprometido para OBJ4. Los escenarios de deriva aislada revelaron, además, que la Tasa de Conformidad —al ser una proporción no ponderada— diluye las violaciones localizadas y puede clasificar como aceptable un sistema con una desviación real. Ese hallazgo motivó un refinamiento del criterio de clasificación que incorpora la severidad del peor hallazgo sin alterar el cálculo de la métrica, y que clasifica correctamente los cinco escenarios. Al margen del experimento, el mecanismo se aplicó sobre una biblioteca de terceros para comprobar su comportamiento ante código real: la regla de integridad de capas resultó exacta, el análisis completo tardó poco más de un segundo y las exenciones declaradas eliminaron cerca de dos tercios de los hallazgos de concentración sin silenciar los ciclos genuinos. El capítulo cerró examinando las amenazas a la validez de la evidencia producida, entre las que destacan la artificialidad de los escenarios, el hecho de que el refinamiento se validara sobre los mismos casos que lo motivaron y el riesgo que introducen las propias exenciones al trasladar a quien configura un juicio que el mecanismo no sabe emitir.

Capítulo 5

Conclusiones

5.1. Conclusiones

Este trabajo se propuso desarrollar y evaluar un mecanismo automatizado que detectara la deriva arquitectónica combinando grafos de dependencia y *Fitness Functions*, con el fin de clasificar el estado de conformidad estructural de proyectos de PyMES sin depender de documentación externa. Los resultados obtenidos permiten afirmar que el objetivo general se cumplió: el mecanismo fue construido, evaluado y demostró ser capaz de discriminar entre una implementación conforme y otra que acumula desviaciones, así como de detectar cada una de las tres familias de deriva cuando se presentan de forma aislada.

El cumplimiento de los objetivos específicos respalda esa afirmación. Respecto a OBJ1, se formuló un catálogo acotado de tres *Fitness Functions* —integridad de capas, ausencia de ciclos y control de concentración de dependencias— en el que cada regla declara su propiedad protegida, su racionalidad, su unidad de verificación, su criterio de incumplimiento y su severidad; el catálogo no reside solo en el documento, sino que el propio prototipo lo deriva de la implementación vigente y puede imprimirlo bajo demanda. En cuanto a OBJ2, se implementó un componente de análisis estático que construye un grafo de dependencias entre paquetes a partir del *bytecode*, prescindiendo por completo de la documentación arquitectónica y conservando, junto a cada arista, la referencia entre clases que la origina. En relación con OBJ3, se construyó un motor de validación que agrega las verificaciones en una Tasa de Conformidad y clasifica el estado del sistema en tres niveles interpretables, empleando los umbrales del 85 % y el 70 %. Por último, OBJ4 se satisfizo mediante un experimento controlado sobre cinco escenarios en el que el mecanismo detectó el cien por ciento de las derivas inducidas sin producir un solo hallazgo sobre la implementación de referencia, superando el criterio de éxito del ochenta por ciento.

La tabla 5.1 recoge esa correspondencia frente a los criterios de éxito que el anteproyecto fijó para cada objetivo, junto con la evidencia concreta que permite verificarla.

Tabla 5.1: Cumplimiento de los criterios de éxito comprometidos en el anteproyecto.

Obj.	**Criterio de éxito comprometido**	**Evidencia de cumplimiento**
OBJ1	Catálogo con al menos una regla ejecutable por familia de deriva, con propósito, evidencia esperada y criterio de incumplimiento.	Tres *Fitness Functions* documentadas en la tabla 3.1, derivadas de la implementación vigente e imprimibles desde el propio prototipo.
OBJ2	Grafo navegable que represente de forma consistente los componentes y dependencias necesarios para evaluar las reglas.	Grafo construido desde el *bytecode* y dibujado por el propio mecanismo, ordenado según la estratificación y con los hallazgos resaltados; las figuras 3.5, 3.6, 3.7 y 4.1 las generó él.
OBJ3	Cálculo automático de la CR, aplicación de los umbrales y clasificación coherente con la matriz de validación.	Ecuación 3.1 y algoritmo 1; clasificación coincidente con la matriz esperada en los cinco escenarios (tabla 4.3).
OBJ4	Detección de al menos el 80 % de las derivas inducidas, línea base sin hallazgos inesperados y reportes comprensibles.	Detección del 100 % de las seis derivas, cero hallazgos sobre la línea base y reportes con evidencia hasta la línea de código (tabla 4.7).

El resultado más instructivo, sin embargo, no fue la confirmación de lo esperado sino el descubrimiento de una limitación. Al someter el mecanismo a escenarios con una única deriva —un diseño que el trabajo incorporó precisamente para medir la sensibilidad de cada regla por separado—, se constató que la Tasa de Conformidad, por ser una proporción no ponderada, diluye las violaciones localizadas hasta el punto de clasificar como aceptable un sistema que contiene una desviación real y documentada. La métrica mide con fidelidad la extensión del deterioro, pero es ciega a su gravedad, y esa ceguera se agrava conforme el sistema crece. La conclusión que de ello se desprende trasciende al prototipo: cualquier indicador de conformidad construido como proporción heredará esta propiedad, de modo que la agregación de hallazgos arquitectónicos no puede descansar únicamente en el conteo.

Los hallazgos dialogan con la literatura existente y la complementan. Frente a los enfoques basados en modelos, que exigen una especificación de referencia (Murphy et al., 2001; Martín et al., 2024; Knodel and Popescu, 2007), la propuesta demuestra que es posible verificar la conformidad infiriendo la estructura directamente del código. Frente a los trabajos centrados en *smells* (Sas et al., 2019; Sas and Avgeriou, 2022), la contribución consiste en traducir la detección en un veredicto agregado que sigue siendo reconstruible por quien lo recibe. Y frente a las propuestas de *Fitness Functions* como práctica (Ford et al., 2017; Paul and Wang, 2019; TNG Technology Consulting, 2026), este trabajo aporta un mecanismo que integra la extracción, la evaluación y la clasificación en un único instrumento ligero. En conjunto, la investigación confirma que la gobernanza arquitectónica continua no es un privilegio exclusivo de las grandes organizaciones, sino una capacidad que puede ponerse al alcance de las PyMES (Modawski and Wolniak, 2025).

5.2. Contribuciones del trabajo

Más allá del cumplimiento puntual de los objetivos, el trabajo aporta cuatro contribuciones que conviene deslindar.

La primera es **conceptual**: la propuesta articula en un único instrumento dos cuerpos de conocimiento que hasta ahora avanzaban por separado —el análisis estático de dependencias y las *Fitness Functions* como práctica de gobernanza— y demuestra que su integración es viable y produce un diagnóstico accionable.

La segunda es **empírica** y no estaba prevista. Comprende dos hallazgos sobre los límites de este tipo de instrumentos, ambos medidos y no supuestos. El primero es que una tasa de conformidad formulada como proporción de verificaciones diluye las violaciones aisladas, y que esa ceguera se corrige combinando la extensión del incumplimiento con la gravedad del peor hallazgo; la corrección es deliberadamente simple —adoptar el más restrictivo de dos veredictos— y por ello sigue siendo explicable ante un equipo, que es la condición que el contexto de las PyMES impone a cualquier instrumento de gobernanza. El segundo es que una regla puramente topológica produce, sobre código real, una proporción de ruido que la vuelve inutilizable si no se le permite conocer el papel de cada paquete: sobre *ArchUnit*, dos tercios de los hallazgos de concentración procedían de dependencias vendorizadas o de un núcleo de anotaciones, y desaparecieron al declarar las exenciones correspondientes sin que los ciclos genuinos se vieran afectados. La lección que se desprende de ambos es la misma y va en contra de la intuición inicial del proyecto: lo que hace útil a un mecanismo de conformidad no es la cantidad de reglas que evalúa, sino la política con que interpreta sus resultados.

La tercera es **metodológica**: el documento ofrece un camino reproducible para construir un mecanismo de verificación de conformidad que no dependa de documentación externa, con decisiones de diseño explícitas, una matriz de validación fijada antes de la ejecución y una estrategia de verificación por etapas que separa el núcleo lógico de su integración con herramientas externas.

La cuarta es **práctica**: el prototipo constituye una base funcional que una PyME puede adoptar, adaptar y calibrar según su contexto —basta con elegir la carpeta del proyecto en su interfaz gráfica para obtener un primer diagnóstico—, cuyo código está estructurado para favorecer la extensión y cuya evidencia se emite en formatos verificables por terceros.

Estas contribuciones dialogan con el estado del arte sin pretender superarlo en todos los frentes. La propuesta no compite con los enfoques basados en modelos en poder expresivo, ni con los enfoques de detección de *smells* en profundidad analítica; su valor reside en el punto de equilibrio que logra entre costo, autonomía e interpretabilidad, precisamente el punto que los trabajos previos dejaban desatendido.

Esta lectura se alinea, sin desviarse, con la brecha que el anteproyecto identificó como espacio de contribución. Allí se constató que ningún antecedente reunía de forma simultánea cinco elementos: el análisis estático exclusivo del código, la combinación de grafo de dependencias con *Fitness Functions*, la clasificación mediante una Tasa de Conformidad, un experimento controlado con criterios explícitos y una orientación específica a las PyMES sin documentación arquitectónica externa. El presente trabajo no añadió objetivos nuevos ni amplió el alcance: se limitó a ocupar, de manera

verificable, ese espacio delimitado, y a documentar con honestidad dónde el instrumento resultó insuficiente y cómo se corrigió.

5.3. Trabajos futuros

La evaluación realizada abre varias líneas de trabajo que conviene explorar, ordenadas aquí por la urgencia que la propia evidencia les confiere.

La más apremiante es la **validación externa** sobre proyectos reales de la industria, con la participación de quienes los mantienen. La aplicación a *ArchUnit* descrita en la sección 4.6 despejó una parte del problema —el mecanismo opera sobre código industrial, con rendimiento holgado y sin degradarse— pero dejó intacta la otra: nadie con conocimiento de ese sistema confirmó si los hallazgos correspondían a deriva real o a decisiones deliberadas, y la arquitectura contra la que se le evaluó fue supuesta por el propio autor. Medir además el efecto de la escala permitiría validar el refinamiento de la clasificación sobre casos independientes de los que lo motivaron, que es la limitación de validez interna que este trabajo no logró contener.

La segunda línea, y la que más acercaría el instrumento a una adopción real, es la incorporación de una **línea base**. El obstáculo que hoy impide aplicarlo al diagnóstico de un sistema legado no es técnico sino de utilidad: una primera ejecución sobre código con deriva acumulada produce un volumen de hallazgos que ningún equipo puede accionar, y el desenlace habitual de esa avalancha es que el instrumento se desactive. La solución conocida consiste en fijar el estado actual como punto de partida y reportar en adelante únicamente las violaciones *nuevas*, de modo que la conformidad se exija sobre el código que se escribe y no sobre el que se hereda. Conviene señalar que esta extensión no añadiría una cuarta *Fitness Function*: el catálogo permanecería en tres, y lo que se incorporaría es una capa de política sobre el motor existente.

La tercera línea es la **reducción del costo de la primera configuración**. Para una organización pequeña, la barrera de entrada no es ejecutar el mecanismo sino redactar la declaración inicial de su arquitectura. Que la estratificación de *ArchUnit* pudiera deducirse de la estructura de sus paquetes y resultara correcta en las sesenta y una verificaciones sugiere que esa declaración es, al menos parcialmente, inferible: un modo que proponga capas candidatas a partir del árbol de paquetes y del sentido de las dependencias, y que fije el umbral de concentración en un percentil de la distribución observada en lugar de un valor fijo, convertiría un ejercicio de análisis en un punto de partida editable. El asistente de catálogo descrito en la sección 3.10 da un primer paso en esa dirección, apoyándose en un modelo de lenguaje en lugar de una heurística fija; queda pendiente evaluar empíricamente la calidad de sus propuestas, por ejemplo contrastándolas con los catálogos que declaren los arquitectos de sistemas reales y midiendo cuántas de sus decisiones se conservan tras la revisión.

A ellas se suman tres extensiones de alcance más acotado: ampliar el catálogo de *Fitness Functions* a otras manifestaciones de la deriva, como la violación de patrones de diseño; refinar la granularidad del análisis para emitir diagnósticos a nivel de clase y no solo de paquete; e integrar el mecanismo en una tubería de integración continua, para lo cual el prototipo ya ofrece la aforDancia de terminar con un código de salida condicionado al veredicto, si bien esa integración solo

resulta sostenible una vez incorporada la línea base, pues de otro modo bloquearía toda entrega desde el primer día. Por último, la extensión a otros ecosistemas mediante herramientas análogas a *ArchUnit* en otras plataformas (Panneerselvam, 2023) ampliaría el alcance de la propuesta más allá del ecosistema Java.

5.4. Lecciones aprendidas

El proceso dejó enseñanzas que trascienden el resultado puntual.

La primera, y la más valiosa, se refiere al diseño experimental: **un único escenario que acumula todas las derivas comprueba que el instrumento reacciona, pero oculta cómo reacciona**. Fue la decisión de construir escenarios con una sola deriva la que reveló la limitación de la métrica; con un solo caso alterado, el trabajo habría concluido que el mecanismo funcionaba correctamente y esa conclusión, aun siendo cierta en sus términos, habría sido engañosa. Aislar las variables no es un refinamiento estético del experimento: es lo que lo convierte en una fuente de conocimiento.

La segunda tiene que ver con la agregación de métricas. Reducir un diagnóstico a una sola cifra facilita la comunicación, pero toda agregación pierde información, y conviene saber cuál. En este caso, la proporción perdía la gravedad de los hallazgos, y recuperarla exigió añadir una segunda dimensión en lugar de complicar la fórmula de la primera. Mantener las dos lecturas separadas y visibles resultó preferible a fundirlas en un índice ponderado que nadie habría podido reconstruir.

La tercera se refiere a la configurabilidad: externalizar las reglas en un archivo de propiedades permitió reutilizar el mismo motor sobre seis sistemas distintos —los cinco escenarios y el propio prototipo— sin modificar una sola línea de código, una decisión de diseño que resultó decisiva.

La cuarta concierne al contrato de las reglas. Una primera versión permitía que los conteos que alimentan la métrica y la evidencia que la explica procedieran de recorridos distintos; obligar a que ambos surjan de una única evaluación eliminó por construcción una clase entera de inconsistencias posibles. En un instrumento cuya credibilidad descansa en que el número y su justificación coincidan, esa garantía no podía quedar en manos de la disciplina del programador.

La quinta se refiere a la validación por etapas —primero el núcleo lógico sobre grafos construidos a mano, después el componente que depende de la herramienta externa—, que redujo el riesgo de confundir un defecto propio con un comportamiento ajeno y permitió ejercitar casos límite que ningún caso de estudio ofrece. Aplicar además el instrumento sobre sí mismo resultó ser una comprobación barata y reveladora: confirmó que la arquitectura descrita era la implementada y, de paso, expuso una limitación de la regla de concentración que ningún escenario diseñado a propósito había mostrado.

Finalmente, una lección sobre el alcance. La disciplina de acotar el catálogo a tres dimensiones, en lugar de perseguir una cobertura exhaustiva, resultó esencial para entregar un instrumento funcional y evaluable. En investigación aplicada, la tentación de ampliar el alcance suele conspirar contra la posibilidad de validar; en este trabajo, la decisión inversa —acotar para poder verificar— fue la que dejó margen para investigar a fondo lo poco que se abarcó, y ese margen fue justamente donde apareció el hallazgo más interesante.

Capítulo 6

Riesgos éticos

6.1. Riesgos éticos

Este capítulo aborda las consideraciones éticas que rodearon el desarrollo y la evaluación del mecanismo. Aunque el proyecto no involucró sujetos humanos ni datos personales, la naturaleza de su artefacto —un instrumento que emite juicios sobre la calidad estructural del código— obligó a examinar con cuidado los usos posibles de sus resultados y las responsabilidades que de ellos se derivan. La exposición conserva la estructura comprometida en el anteproyecto: primero los principios que orientaron el diseño, después los cinco riesgos identificados con el tratamiento asignado a cada uno, y por último el estado en que quedaron tras la ejecución del proyecto.

6.2. Principios orientadores

El desarrollo y la evaluación del mecanismo se rigieron por cuatro principios.

El primero fue la **privacidad por diseño**: el mecanismo opera sobre el código compilado y de manera local, sin transmitir, almacenar ni exponer información fuera del entorno de quien lo ejecuta; la única excepción es el asistente de catálogo, que es opcional, se activa a petición expresa y puede usar un modelo local. El segundo fue la **transparencia del diagnóstico**: toda clasificación emitida se acompaña de la evidencia concreta que la sustenta, de modo que el resultado sea auditable. El tercero fue la **supervisión humana**: el instrumento se concibió como un apoyo a la decisión, nunca como un reemplazo del juicio de los profesionales. El cuarto fue el **uso proporcional de las métricas**: las mediciones se limitaron a propiedades estructurales del código y se evitaron inferencias sobre el desempeño de personas o equipos.

6.3. Riesgos identificados y tratamiento aplicado

Siguiendo la lógica de gestión de riesgos adoptada en el anteproyecto, cada situación recibió uno de cuatro tratamientos posibles —evitarla, mitigarla, transferirla o aceptarla de forma controlada—, y a continuación se describe cómo se materializó ese tratamiento durante la ejecución.

- **Riesgo sobre información sensible. Tratamiento: evitarlo.** El análisis estático requiere acceso al código, por lo que la respuesta adecuada consistió en impedir desde el inicio todo escenario innecesario de exposición. El prototipo se ejecuta íntegramente en local, sin servicios externos ni telemetría, y los artefactos que genera contienen únicamente relaciones estructurales y métricas agregadas. La evaluación se realizó sobre sistemas sintéticos construidos

para el proyecto, sobre el propio prototipo y sobre una biblioteca de código abierto de acceso público, de modo que en ningún momento se procesó información sujeta a confidencialidad.

- **Riesgo de sesgo algorítmico y opacidad del diagnóstico. Tratamiento: mitigarlo.** Para evitar decisiones de refactorización basadas en umbrales arbitrarios o interpretaciones erróneas, las reglas se mantuvieron explícitas, versionadas y trazables: residen en un archivo de configuración legible, y cada hallazgo declara la regla activada, la evidencia estructural encontrada —hasta el método y la línea que originan la dependencia— y la severidad asignada. La Tasa de Conformidad se calcula como un cociente explícito que cualquier miembro del equipo puede rehacer a mano, y no mediante un modelo cuyo resultado no pudiera reconstruirse. La validación se apoyó, además, en una matriz de comportamiento esperado fijada antes de la ejecución y sometida a revisión humana.

- **Riesgo de impacto sociotécnico en los equipos. Tratamiento: mitigarlo.** Para reducir la posibilidad de que la gobernanza automatizada se perciba como vigilancia punitiva, el mecanismo mide exclusivamente propiedades de paquetes y no atribuye hallazgos a personas: ni el modelo de datos ni el reporte contemplan la noción de autor. El instrumento se propone como soporte para conversaciones técnicas, retrospectivas arquitectónicas y priorización de refactorización, no como insumo de evaluación individual.

- **Riesgo de formalismo vacío. Tratamiento: aceptarlo de forma controlada.** Siempre existirá la posibilidad de que una métrica de conformidad se use de manera mecánica y sin reflexión contextual. Ese riesgo no puede eliminarse, por lo que se aceptó de forma controlada y se redujo su impacto por dos vías. La primera es que el reporte declara siempre su propio alcance —qué reglas se evaluaron, qué umbrales se aplicaron, qué paquetes quedaron fuera de la estratificación y qué aspectos no cubre el diagnóstico—, advirtiendo de forma expresa que un veredicto *aceptable* significa que no se hallaron incumplimientos de esas reglas y no que el sistema carezca de problemas estructurales. La segunda es la configurabilidad de las reglas y los umbrales, que obliga a cada organización a decidir explícitamente qué considera admisible en lugar de heredar un criterio ajeno.

- **Riesgo de sobreinterpretación de la Tasa de Conformidad. Tratamiento: transferirlo parcialmente a la decisión humana responsable.** La CR representa únicamente el cumplimiento de las reglas configuradas dentro del alcance del prototipo. La evaluación aportó evidencia concreta de por qué esta advertencia no era retórica: en dos de los escenarios evaluados, la métrica clasificó como aceptable un sistema que contenía una violación real, precisamente porque una proporción diluye los incumplimientos localizados. El hallazgo confirma que la cifra no puede leerse como veredicto absoluto y que la decisión final sobre priorización de refactorización o aceptación del riesgo corresponde al responsable técnico y no a la herramienta. En respuesta, el mecanismo incorporó la severidad del peor hallazgo como criterio complementario y advierte de forma explícita en el reporte cuando ese criterio ha endurecido el veredicto que la métrica habría emitido por sí sola, de modo que la discrepancia entre ambas lecturas quede a la vista de quien decide.

- **Riesgo de exposición y de propuestas erróneas del asistente de catálogo. Tratamiento: mitigarlo.** El asistente descrito en la sección 3.10 introduce dos riesgos que el mecanismo no tenía. El primero es la salida de información hacia un tercero: se mitigó enviando solo el perfil estructural —nunca el código fuente—, mostrando su texto exacto antes de enviarlo, admitiendo modelos que se ejecutan en el propio equipo y no conservando la clave de acceso salvo petición expresa. El segundo es que un modelo de lenguaje proponga un catálogo erróneo o legitime la deriva existente: se mitigó instruyéndolo para declarar la arquitectura prevista y señalar la deriva en lugar de permitirla, validando la propuesta con el mismo cargador que cualquier catálogo, mostrando su diagnóstico antes de adoptarla y reservando la adopción a la persona.

La tabla 6.1 sintetiza los seis riesgos, su tratamiento y el estado en que quedaron al cierre del proyecto.

Tabla 6.1: Riesgos éticos, tratamiento aplicado y estado al cierre del proyecto.

Riesgo	**Tratamiento**	**Control materializado**	**Estado**
Información sensible	Evitar	Ejecución local sin servicios externos; evaluación sobre sistemas propios, sintéticos y de código abierto.	Controlado
Sesgo y opacidad del diagnóstico	Mitigar	Reglas explícitas y versionadas; evidencia hasta la línea de código; métrica reconstruible a mano.	Controlado
Impacto sociotécnico en los equipos	Mitigar	El modelo de datos no contempla la noción de autor; los hallazgos se atribuyen a paquetes.	Controlado
Formalismo vacío	Aceptar de forma controlada	Declaración de alcance en todo reporte; umbrales y reglas de decisión configurables.	Residual
Sobreinterpretación de la CR	Transferir a la decisión humana	Severidad como criterio complementario; aviso explícito cuando ambos criterios discrepan; declaración de exclusiones.	Residual
Exposición y propuestas erróneas del asistente	Mitigar	Solo perfil estructural, visible antes de enviarse; modelos locales; propuesta validada, previsualizada y adoptada por la persona.	Controlado

6.4. Riesgos residuales

Dos riesgos permanecen activos en un nivel residual y conviene declararlo sin atenuantes.

El primero es el formalismo vacío. Ningún control técnico impide que una organización convierta la Tasa de Conformidad en un objetivo por sí mismo y persiga el número en lugar de la propiedad que representa. Las salvaguardas incorporadas —la declaración de alcance y la configurabilidad— reducen la probabilidad de que ello ocurra por desconocimiento, pero no por decisión deliberada.

El segundo es la interpretación indebida de los resultados en contextos arquitectónicos distintos del evaluado. El autoanálisis del prototipo mostró un caso concreto: la regla de concentración

señala como problemáticos a componentes que concentran dependencias por diseño, como una raíz de composición o una fachada. En un sistema real, un hallazgo así podría desencadenar una refactorización innecesaria si se acatara sin criterio. La mitigación disponible es la que el proyecto ya adopta —transparencia del diagnóstico y recomendación explícita de supervisión humana—, y la solución de fondo, el registro de excepciones justificadas, queda planteada como trabajo futuro.

Ambos riesgos se consideran aceptables en el estado actual del proyecto, dado que el artefacto se entrega como prototipo de investigación y no como producto de adopción industrial. No se identificaron conflictos de interés ni implicaciones éticas no previstas durante la ejecución.

Bibliografía

Anthony, E., Berntsson, A., Santilli, T., and Wohlrab, R. (2024). We're drifting apart: Architectural drift from the developers' perspective. In *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA 2024)*, pages 101–111.

AOE Technology Radar (2026). Architecture decision records (ADR) — methods & patterns. Consultado en 2026.

Bass, L., Clements, P., and Kazman, R. (2012). *Software Architecture in Practice*. Addison-Wesley Professional, Boston, MA, USA, 3rd edition.

codecentric AG (2026). Archunit in practice: Keep your architecture clean. Consultado en 2026.

Continuous Architecture (2026). Fitness functions – safeguard architecture with automated checks. Consultado en 2026.

Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30. ACM.

de Silva, L. and Balasubramaniam, D. (2012). Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151.

Diaz-Pace, J. A., Soria, A., Rodriguez, G., and Campo, M. R. (2011). Taking archsync to the real world: An analysis of three case-studies.

Ford, N., Parsons, R., and Kua, P. (2017). *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media, Sebastopol, CA, USA.

Gansner, E. R., Koutsofios, E., North, S. C., and Vo, K.-P. (1993). A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230.

Hamed, A. S. E., Elbakry, H. M., Riad, A. E., and Moawad, R. (2023). A proposed technical debt management approach applied on software projects in egypt. *Journal of Internet Services and Information Security (JISIS)*, 13(3):156–177.

Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–105.

Jamil, J. (2026). Zero trust architecture for small/medium enterprises in hybrid cloud: A lightweight blueprint. *Scholars Journal of Engineering and Technology*, 14(1):48–56.

Jolak, R., Karlsson, S., and Dobslaw, F. (2025). An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software*, 225:112382.

Klein, J., Nord, R., and Shull, F. (2022). Why architecture conformance matters for software systems. Technical Report DM22-0924, Software Engineering Institute, Carnegie Mellon University.

Knodel, J. and Popescu, D. (2007). A comparison of static architecture compliance checking approaches. In *Sixth Working IEEE/IFIP Conference on Software Architecture (WICSA 2007)*, page 12. IEEE.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076.

Manglaras, O., Farkas, A., Woolford, T., Treude, C., and Wagner, M. (2026). Modaro: A modular approach to architecture reconstruction of distributed microservice codebases. *arXiv preprint arXiv:2602.08181*.

Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, Upper Saddle River, NJ, USA.

Martín, D. S., Camargo, V. V. D., and Angulo, G. (2024). A mape-k-based method for architectural conformance checking in self-adaptive systems. Trabajo relacionado con verificación de conformidad arquitectónica en sistemas autoadaptativos.

Modawski, A. and Wolniak, A. (2025). Lightweight AI governance (LAIG) framework for SMEs. In *Proceedings of the European Workshop on Trustworthy AI (TRUST-AI 2025), organized as part of the European Conference of Artificial Intelligence (ECAI 2025)*, Bologna, Italy. CEUR-WS.org.

Murphy, G. C., Notkin, D., and Sullivan, K. J. (2001). Software reflexion models: Bridging the gap between source and high-level models. *IEEE Transactions on Software Engineering*, 27(4):364–380.

Niessen, L. (2026). Fitness functions: Automating your architecture decisions. Consultado en 2026.

Okoone (2026). Make your architecture agile with fitness functions — okoone. Consultado en 2026.

Panneerselvam, S. (2023). Maintaining architectural integrity in .net projects with NetArchTest. Consultado en 2026.

Patrício, L. and Varela, L. (2025). Implementation of a low-cost digital transformation model for small- and medium-sized industrial enterprises in the context of industry 4.0. *Sci*, 7(4):187.

Paul, P. and Wang, R. (2019). Fitness function-driven development. *Thoughtworks*.

Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Perry, D. E. and Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52.

Pigazzini, I. (2021). *Evaluating and Detecting Architecture Erosion*. Phd thesis, Università degli Studi di Milano-Bicocca.

Rosik, J. (2007). Maintaining architecture consistency. Tesis, University of Limerick.

Rossini, S. (2022). Agile architecture part 7: Fitness functions & architectural guardrails. Consultado en 2026.

Runeson, P. and Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164.

Sas, D. and Avgeriou, P. (2022). An architectural technical debt index based on machine learning and architectural smells. Bernoulli Institute, University of Groningen.

Sas, D., Avgeriou, P., and Fontana, F. A. (2019). Investigating instability architectural smells evolution: An exploratory case study. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 557–567. IEEE.

Sugiyama, K., Tagawa, S., and Toda, M. (1981). Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160.

TNG Technology Consulting (2026). Unit test your java architecture - archunit. Consultado en 2026.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg.

Anexo A

Reproducción del experimento

Este anexo reúne la información necesaria para reproducir íntegramente el experimento descrito en el capítulo de evaluación. Se incluye porque una evaluación cuyos resultados no pueden rehacerse por un tercero aporta poca evidencia, con independencia de lo cuidadoso que haya sido su diseño.

A.1. Entorno y dependencias

El prototipo se implementó con nivel de lenguaje Java 17 y se verificó sobre JDK 17 y JDK 26. La única dependencia externa es *ArchUnit* en su versión 1.3.0, empleada exclusivamente para importar el *bytecode* y resolver las dependencias entre clases; el resto de la lógica —el algoritmo de detección de ciclos, el motor de conformidad y los exportadores— no depende de bibliotecas de terceros.

La biblioteca se distribuye junto al repositorio, de modo que la compilación y la ejecución no requieren Maven, repositorio de artefactos ni conexión a internet. Esta decisión responde al requisito de ligereza (RNF2) y, de paso, hace que el experimento siga siendo reproducible aunque los repositorios remotos cambien. Todas las operaciones se invocan mediante un único lanzador, `arquitecta.bat` en Windows o `./arquitecta.sh` en macOS y Linux, que compila el prototipo la primera vez y cada vez que su código cambia.

A.2. Estructura del repositorio

```
prototipo/
  arquitecta.bat / .sh        lanzador unico (aplicacion y tareas)
  src/main/java/              nucleo, servicio, interfaz y asistente
  src/test/java/              bateria de pruebas (234 comprobaciones)
  lib/                        ArchUnit 1.3.0 y slf4j-api
  caso-estudio/
    c0-base/                  escenario C0 (linea base)
    c1-capas/                 escenario C1 (ruptura de capas)
    c2-ciclo/                 escenario C2 (ciclo de dependencia)
    c3-concentracion/         escenario C3 (componente dios)
    c4-combinado/             escenario C4 (las tres derivas)
    rules.properties          catalogo de reglas del caso de estudio
  ejemplos/
    autoanalisis/             catalogo para el autoanalisis
```

```
    archunit/                configuraciones de la seccion 4.6
    graficos/                grafos en SVG generados por la herramienta
    interfaz/                capturas de la interfaz grafica
  scripts/                   guiones que invoca el lanzador
```

Cada escenario tiene la estructura de un proyecto Maven: su código está en `src/main/java` y la compilación deja sus clases en `target/classes`, de modo que la carpeta del escenario puede indicarse tal cual, desde la línea de comandos o desde la interfaz gráfica.

A.3. Pasos para reproducir el experimento

1. **Compilar.** Ejecutar `arquitecta compilar` (`./arquitecta.sh compilar` en macOS o Linux). El guion compila el núcleo y la batería de pruebas en `out/`, empaqueta la herramienta en `out/arquitecta.jar` y compila cada escenario en su `target/classes`. El paso es opcional: los siguientes compilan por su cuenta si hace falta.

2. **Verificar el núcleo.** Ejecutar `arquitecta pruebas`. La batería debe reportar 234 comprobaciones superadas y terminar con código de salida cero; cualquier fallo detiene la reproducción y señala el caso concreto que discrepa.

3. **Ejecutar el experimento.** Ejecutar `arquitecta experimento`. El guion imprime el catálogo de reglas, recorre los cinco escenarios bajo los dos modos de clasificación y deposita la evidencia en `resultados/`.

4. **Contrastar.** Comparar el archivo `resultados/modo-cr/resumen.csv` con la tabla 4.2 y el archivo `resultados/modo-cr-severidad/resumen.csv` con la tabla 4.6.

Para reproducir el autoanálisis descrito en el capítulo de desarrollo basta con dirigir el mecanismo hacia su propio directorio de clases compiladas:

```
arquitecta analizar out\arquitecta-core ^
     ejemplos\autoanalisis\rules-arquitecta.properties
```

A.4. Figuras de grafos y sistema externo

Las figuras de grafos de este documento se regeneran con `arquitecta figuras`, que escribe las versiones TikZ en el directorio `figuras/` de la monografía y las SVG en `ejemplos/graficos/` del prototipo. Para obtener solo el dibujo de un sistema cualquiera basta con indicar las opciones `--svg` o `--tikz`:

```
arquitecta analizar lib\archunit-1.3.0.jar ^
     ejemplos\archunit\rules-archunit.properties --svg archunit.svg
```

El mismo análisis puede hacerse desde la interfaz gráfica, que se abre ejecutando el lanzador sin argumentos —o con doble clic sobre él— y permite elegir el `.jar` y el catálogo con un selector de archivos; su pantalla de inicio ofrece, además, los cinco escenarios del caso de estudio para analizarlos con un clic. El ejemplo reproduce además el análisis de la sección 4.6: el mecanismo admite directamente el archivo `.jar`, y el directorio `ejemplos/archunit/` contiene las configuraciones con y sin exenciones cuyos resultados recoge la tabla 4.8.

A.5. Evidencia generada

Cada ejecución con la opción `--salida` produce cuatro artefactos por escenario y una fila en el resumen comparativo:

- un **reporte legible** (`.txt`) con la Tasa de Conformidad, los estados deducidos por cada criterio, los hallazgos con su evidencia y la declaración de alcance del diagnóstico;
- la **evidencia estructurada** (`.json`), que incluye el grafo completo, los conteos por regla y la totalidad de las trazas de cada hallazgo, sin el recorte que el reporte legible aplica por brevedad;
- el **grafo de dependencias** dibujado en tres formatos con la misma disposición: una imagen `.svg` que se abre en cualquier navegador, una figura `.tikz` lista para incluirse en LaTeX y un archivo `.dot` para quien prefiera procesarlo con Graphviz;
- los **hallazgos tabulados** (`.csv`), una fila por hallazgo, para su tratamiento en una hoja de cálculo.

El archivo `resumen.csv` de cada modo agrega una fila por escenario con el número de paquetes, dependencias, verificaciones e incumplimientos, la Tasa de Conformidad y los tres estados calculados. Es el artefacto del que proceden directamente las tablas del capítulo de evaluación.

A.6. Determinismo de los resultados

El mecanismo no emplea aleatoriedad, concurrencia ni marcas temporales en su cálculo, por lo que ante la misma entrada produce siempre el mismo diagnóstico. La construcción del grafo recorre las clases y sus dependencias en orden alfabético, y las estructuras que lo recorren conservan el orden de inserción, de modo que también el orden de los hallazgos y de su evidencia es estable entre ejecuciones y no depende de cómo se nombre la ruta analizada ni del orden en que la biblioteca de importación entregue las clases. Ello permite comparar dos corridas mediante una simple diferencia textual de sus archivos de evidencia, sin necesidad de normalización previa. Las capturas de la interfaz gráfica son la única excepción: dependen de las fuentes tipográficas del equipo y muestran el tiempo de análisis, por lo que se regeneran junto con las figuras pero no se comparan byte a byte.

Al ejecutar sobre JDK 26 aparecen dos avisos en la salida de error —la ausencia de un proveedor de *logging* para SLF4J y una advertencia sobre el uso de `sun.misc.Unsafe` emitida por una biblioteca embebida en ArchUnit— que no afectan al resultado y no se presentan con JDK 17.